



UNIVERSITY OF
LINCOLN

SCHOOL OF MATHEMATICS
AND PHYSICS

**Prediction of rail traffic at level
crossings via time series
classification**

Supervised by Dr. Matthew Watkins

May 20, 2020

Scientific Report in the 3rd Year Module
MTH3005M Mathematics Project

Abstract

There are many internet of things (IoT) devices which make it possible to observe some variables in real time for an object, e.g. using one or more sensors. This brings the issue of how the observed values can be classified and whether the next set of observations can be accurately predicted. With the exponential rate of evolution of supervised learning algorithms, it is now possible to efficiently and effectively collect, store, process and analyze high volumes of data on parallel architectures, allowing for high accuracy predictions whilst keeping computation time short. The aim of this project is to discuss the viability of introducing machine learning models into navigation technologies, supported by a thorough development of such a model. Specifically, the proposal is developing and deploying classifier models into map applications to predict rail crossing closures ahead of a driver, though the outlined method is applicable in general to many different applications of time series analysis and forecasting. The project also aims to compare popular classifiers to benchmark their performance, also offering a short discussion of the suitability of the models for deployment into a live system. Finally, the document offers a critique of the developed method and possible points of improvement for the model, some of which were out of scope for the purpose of this project.

Contents

1	Introduction	4
2	Prerequisite understanding	6
2.1	Nearest Neighbour	6
2.1.1	Choosing value of k	7
2.2	Random Forest	9
2.2.1	Hyperparameters	9
2.2.2	Measuring the quality of a split	10
2.3	Neural Network	16
2.3.1	Propagation	17
2.3.2	Backpropagation	17
2.3.3	Training	17
2.3.4	Activation function	18
2.4	One-hot encoding	19
2.5	N-Fold Cross validation	19
3	Method	21
3.1	Overview	21
3.2	Data Collection	21
3.2.1	Web scraping	22
3.2.2	Manual data collection	22
3.3	Simulation	24
3.4	Data Preparation	27
3.4.1	One-hot	27
3.4.2	Dealing with timestamps	27
3.4.3	Feature engineering	28
3.4.4	Binary vs multi class classification	29
3.4.5	Time lag	30
4	Results	32
4.1	Classification process	32
4.1.1	Performance assessment	33
4.1.2	Reducing sample size	34
4.1.3	Nearest Neighbour	36

4.1.4	Random Forest	38
4.1.5	Neural Network	40
4.2	Discussion	42
4.2.1	Best model	42
4.2.2	Methods of improvement	43
5	Conclusion	46
6	Acknowledgements	48
7	Appendix	49
7.0.1	GitHub repository	49
7.0.2	Collected data	49
7.0.3	Research document	51
7.0.4	Logbook 1	56
7.0.5	Logbook 2	61
7.0.6	Logbook 3	75
	Bibliography	96

Chapter 1

Introduction

The use of IoT devices supported by machine learning plays an important role in data driven decision making - such a model has the ability to provide predictions in real time, based on real, accurate and live data. The ability to reliably predict future values based on some observed samples is possibly one of the most exciting challenges within the field of machine learning. When applied on time series data, problems are classified as forecasting and time series analysis problems, where the goal is to predict the value of some observable variable y at time t or the value of t at which $y = x$.

Classifiers are a category of supervised learning algorithms [17] which build a mathematical model of a supplied data set Y to make predictions $\hat{y}$ in the future by finding some function f . The other category is regression, which is unfortunately out of the scope for this project, despite time series also being suitable for regressors in certain cases like predicting air pollution [15, 13] or rainfall [33]. A built model will unveil relationships in the data to make the predictions; generally, the more **accurate** the data is, the stronger the predictions will be. In many cases, some form of feature engineering is essential as part of pre-processing, otherwise classifiers may find false relations in the data.

Supervised learning algorithms have typically been used for weather forecasting [84], stock market [62] and in medicine [89]. The quick execution time of such models are usable through cloud technologies [65], which makes their use very lucrative to most sectors.

Whilst popular map applications started including public transport timetables and road traffic information since 2007 [102], rail crossings are not recognised as an obstacle or a delay to a journey despite the crossings having the potential of significantly affecting one's overall travel time. Including rail traffic information in map applications could help to foresee the traffic ahead (e.g. at upcoming level crossings), estimate the extra time for the journey and then issue reroutes in time if needed.

This paper places the focus on using classification techniques on time series data to produce a model which can predict rail traffic, that could theoretically be implemented in a real time navigation application. It explains the production of a multi-class classifier which answers the question "is the crossing traversable at t ?; if not, how long for?". By being able to answer for current time t , it can also be answered for $t + 1$, $t + 2$, ..., $t + k$ as well, hence being able to provide a forecast to some degree. The paper can be used as a guide for solving a time series based classification problem in general, as the methods outlined here can rigorously be applied to other problems where some event E happens at time t , or the value y of something is x at time t .

Before diving straight into the method, Chapter 2 outlines preliminary knowledge that is required for understanding the developed method - the idea behind classifiers which are explored as well as some techniques that are applied during data preparation. Chapter 3 describes the motivation and the execution of building such a classifier step by step, pointing out ways of improvement at times. The results achieved by the method are then described and assessed in Chapter 4, also offering a discussion of the obtained results and ways to improve the model(s) further. Chapter 5 then argues the strongest and most suitable model along with some additional statistics based on the collected data.

Chapter 2

Prerequisite understanding

It is essential to first lay down preliminary knowledge of the techniques and algorithms used, their effectiveness and reason for their use. The classifiers discussed in this chapter have been selected to build a model for the purpose of this project. In general, one should try multiple classifiers - usually with several set of hyperparameters - to ultimately come up with one that works the best according to the selected metric(s). In some cases, it is not possible to increase a classifier's effectiveness without changing the data structure [50]; in most instances, the best model is discovered by trying different sets of hyperparameters [10, 9]. To evaluate a model effectively, forms of cross validation [86] should be employed, also covered in this chapter.

2.1 Nearest Neighbour

Nearest neighbour operates on the logic that when data points are plotted, those belonging to the same class are likely to be near each other [35]. It assigns the class label to a test sample based on its k neighbours [69, 3]. If $k = 1$, the test sample is assigned the class of the absolute nearest neighbour, and for $k > 1$ the majority class of the k nearest neighbours is assigned.

Let there be n training samples, each with features m and a classification label of the sample c , $s = [x_0, x_1, \dots, x_m, c]$. Let the objective function be the Euclidean distance

$$d(x, s) = \sqrt{(X_1 - S_1)^2 + (X_2 - S_2)^2}.$$

If a test sample is x and the training samples are $S = \{s_0, s_1, \dots, s_n\}$ then we calculate $d(x, s)$ for each $s \in S$, and sort the results in ascending order. Take the first k classification labels, c , and assign to x the most common value of c [95].

Example. Consider the following data set with arbitrary x and y values and a classification c that has labels *True* and *False* with $k = 3$. Now some other

Table 2.1: Nearest Neighbour Example data set

x	y	c
1	2	True
1	3	True
2	6	False
1	5	False
3	1	True
2	1	True

data points, (5, 6) and (3, 5) are gathered without any classification labels and the task is to determine which category they belong to. It just so happens that

Table 2.2: Distance of data set to unseen data

x	y	Distance from (5, 6)	Distance from (3, 5)
1	2	$\sqrt{32}$	$\sqrt{13}$
1	3	$\sqrt{10}$	$\sqrt{1}$
2	6	$\sqrt{9}$	$\sqrt{2}$
1	5	$\sqrt{17}$	$\sqrt{4}$
3	1	$\sqrt{29}$	$\sqrt{16}$
2	1	$\sqrt{34}$	$\sqrt{17}$

the same 3 data points are the closest neighbours to both new points which are going to be doing the voting. Therefore both data points are classified to belong

Table 2.3: Classification of the points

x	y	Vote for (5, 6)	Classification	Vote for (3, 5)	Classification
1	3	<i>True</i>	False	<i>True</i>	False
2	6	<i>False</i>		<i>False</i>	
1	5	<i>False</i>		<i>False</i>	

to the category **False**.

2.1.1 Choosing value of k

The most significant hyperparameter is value of k , the number of closest neighbours of a training sample that partake in the voting process. The most optimal value of k is context specific and therefore one should test a wide range of candidates. In general, as $k \rightarrow 1$, the stability of the predictions decreases.

Example. Consider a simple example with $n = 10$ as depicted by Figure 2.1:

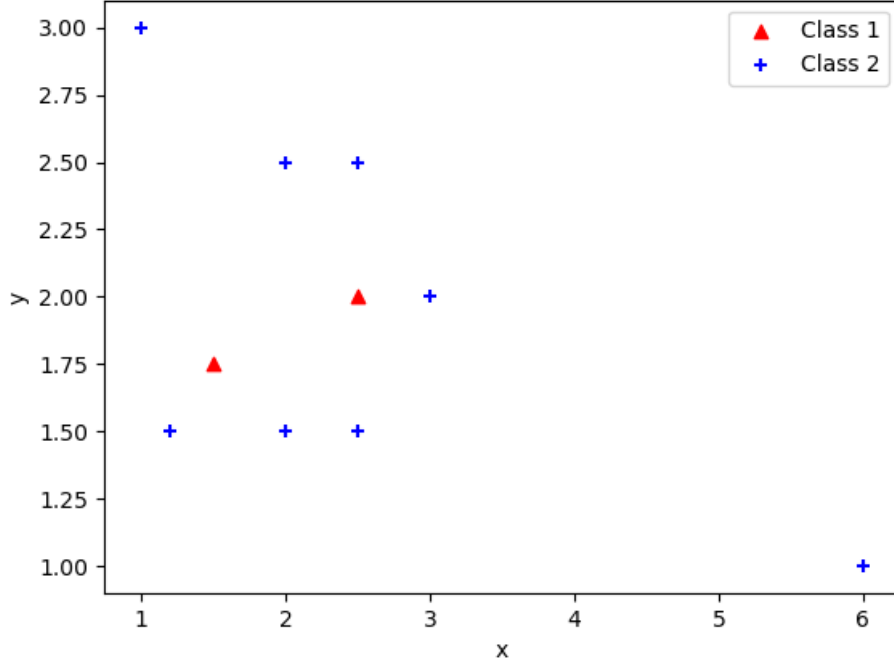


Figure 2.1: Nearest Neighbour stability example with $n = 10$

Consider that from the data set of 10 points, 2 belong to Class 1 with the remaining belonging to Class 2. For $k = 1$, the point $(3, 2)$ is incorrectly classified to be Class 1 because $(2.5, 2)$ is the one closest point, despite the candidate being almost only near Class 2 samples. This shows the general problem with $k \rightarrow 1$, but also what can happen with an even number of k . Choosing $k = 2$ for this example, the nearest neighbours are $N = \{(2.5, 2), (2.5, 1.5), (2.5, 2.5)\}$ with N_1 and N_2 being the same distance away from the candidate. As a result of the points being the same distance away from $(3, 2)$, whichever comes first in the sorting will be picked as the nearest neighbour, in this case N_1 . Now the problem is that there is exactly 1 vote for Class 1 and exactly 1 for Class 2 which results in the same resolution: the one that happens to come earlier in the sorting process is picked. From N_0 and N_1 , N_0 would be picked purely because it comes first in the ascending sort due to its y component being closer to 0. Albeit giving a correct classification in this instance, this again shows unstability when the value of k is poorly chosen.

The inverse is also true. Increasing the value of k increases the stability of predictions due to majority voting, but only up to a certain point. It is beneficial to monitor the chosen metric values over several **odd** values of k , identify

where the classifier’s accuracy starts dropping and choose the suitable value accordingly.

2.2 Random Forest

Random forest works by constructing many decision trees that each make their individual predictions, carry out majority voting and the input is then classified by the label that has the most votes [14]. Bootstrap aggregation, or bagging, is a concept where a random number of samples are selected from the original set with replacement, making the trees correlate less - the trees need to correlate with each other as little as possible [78] so that each tree behaves differently and thus increasing the accuracy [21].

Algorithm 1 Growing a random forest

Considering N as the number of training samples, T_b a tree, m the number of chosen features out of total p features, the algorithm to grow a forest of many trees in general is:

1. Draw a bootstrap sample of size N
 2. Grow a tree on the bootstrapped data, by recursively carrying out the operations for each node of the tree until a stopping condition is met (e.g. minimum node size):
 - (a) Select m random variables
 - (b) Pick the best variables among m to split via $Gain(S, F)$
 - (c) Split the node into two daughter nodes (unless pure)
 3. Output all trees of the forest
-

In the following subsections, the algorithm is explored more in-depth, considering the available hyperparameters and their effect on the overall performance.

2.2.1 Hyperparameters

In general, hyperparameters all have lower and upper bounds, outside of which they start negatively affecting the performance of the forest [78]. There are several hyperparameters that could either influence the complexity of a forest, serve as a stopping condition or impact accuracy. Consider those with the biggest impact on performance [60, 78]:

1. **n-estimators** - the number of decision trees constructed,
2. **max-features** - the number of features to be considered while deciding a split,

3. **max-depth** - the number of levels a tree is allowed to have,
4. **min-samples-split** - defining how many samples a node must contain before it can be split,
5. **min-samples-leaf** - defining the number of samples a leaf node must have **at least**

This is the list of the most commonly adjusted parameters, which all have the largest effect on the accuracy of a model.

n-estimators. The number of decision trees used for the voting process. The more trees there are, the more accurate a model becomes as stated by Liaw & Wiener [63], but high values are computationally expensive, making the building process slow.¹

max-features. Increasing this value may also increase the performance of the model, because it can now select more features to split from [78]. However, it also makes the trees less diverse; intuitively, lowering this value means that the trees will correlate less and produce more stable results [78] which may be worse on average than with a higher value [78]. It is then essential to find the optimal value that neither under nor overfits the model. By default, packages typically use $\sqrt{p}$ [75] where p is the number of total features.

max-depth. When a tree makes a large number of splits (and therefore has a large depth), it has the potential to explain the variation of the data, but this may also result in overfitting. Note that in certain cases, this parameter should be untouched so that trees can make splits until their leaves are pure,² as intended.

min-samples-split. Defining the number of samples required for a split can be very bad for a model, because there is the potential that nodes do not achieve purity due to insufficient samples (and thus splits). This value therefore should be based on the size of the training data set [78].

min-samples-leaf. Similarly to the previous parameter, large values can cause a setback for the model rather than the opposite. With too high of a value, the trees may not be able to capture sufficient patterns within the data due to being unable to split enough times [78]. This therefore depends on the size of the training set and requires tuning [78].

2.2.2 Measuring the quality of a split

Decision trees are constructed such that the most significantly separating features and decisions are at the top of the tree - in other words, information gain is computed for each feature and the one yielding the best value is picked first. The information gain of each remaining feature is computed again, choosing the highest value which determines the most significant split. Once a feature

¹Breiman [14] stated that using more trees than required may be unnecessary but does not harm the model.

²contains nothing but instances belonging to one certain class

is selected to be operated on, it must be further split into two daughter nodes, unless the set at this point is **pure**. The quality of a split is measured either via computing its entropy [83, 5] or Gini impurity [40]. Both methods calculate and convey the same information, however the calculation of entropy is slower due to $\log$ [6]. It only matters in a minority (2%) of cases for accuracy whether Gini impurity or entropy is used [82]; let's overview each criterion briefly.

Let S denote the set of inputs, each with n features, and define entropy as

$$Entropy(p) = - \sum p_i \log_2 p_i$$

where p_i is the frequentist probability of a class in the set (for binary case, $i = 2$). Let F denote a feature that has values $f = [f_1, f_2, \dots, f_n]$, $|S_{f1}|$ be the number of elements in the feature with value f_1 and $|S|$ to be the size of the set. Then, the information gain is

$$Gain = Entropy(S) - \sum \frac{|S_f|}{|S|} Entropy(S_f)$$

. The Gini index is defined as

$$G = \sum p_i * (1 - p_i)$$

which is calculated initially for the dataset G_{init} , then once for each daughter node G_l and G_r . Let $|S_l|$ denote the number of samples in the left branch and $|S_r|$ in the right branch. The Gini Gain for a split is then given by

$$Gain = G_{init} - \frac{|S_l|}{|S|} G_l - \frac{|S_r|}{|S|} G_r$$

Since Gini is a much simpler criterion to understand and use manually, the following example is worked using the Gini criterion. Traditionally, this criterion is used by default in packages [75] because of the computational edge it has over entropy.

Example. Consider the following dataset depicted by Figure 2.2, where x and y are arbitrary attributes. Notice that the three groups of classes can easily be separated by two straight lines. The algorithm tries to find the most optimal x and y coordinates to draw separating lines through, which will separate each class the most. To find the most optimal split by brute force, one selects all features individually, trying all of their possible threshold values and then decides on the best feature and split condition. Brute forcing is not optimal, considering that the large amount of combinations available make it a slow process. In the case of this example brute forcing allows readers to understand splitting intuitively.

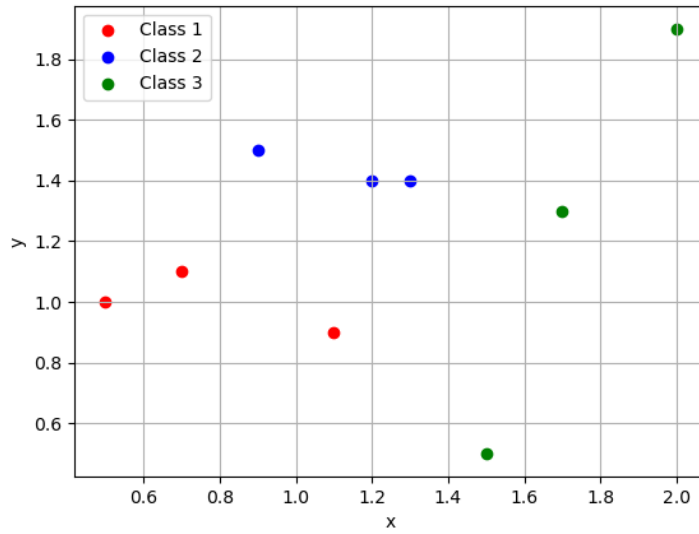


Figure 2.2: Example of Random Forest Classifier with 3 classes

Let x be the first splitting feature. Figure 2.3 outlines the threshold values for this feature, i.e. the values that the algorithm may try calculating the gain of.

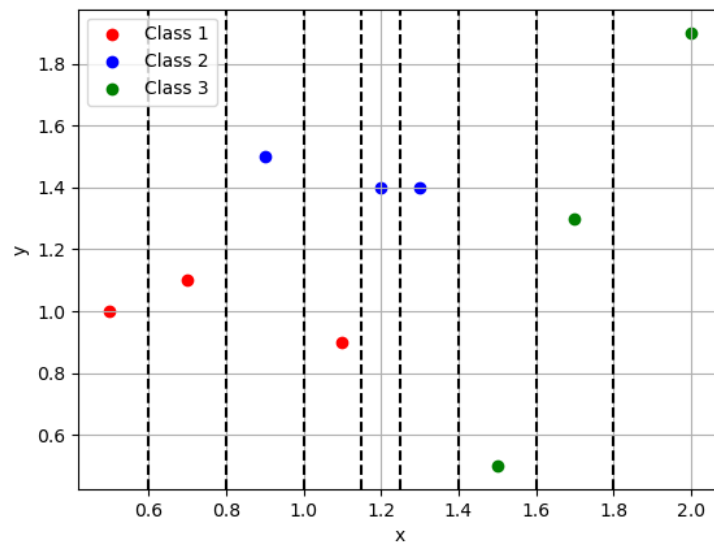


Figure 2.3: Threshold values for x

Let us select $x = 0.6$ and calculate its Gini impurity. If this condition is

selected, there is 1 (red) sample in the left branch, and the rest of the samples are in the right branch (3 blue, 3 green and 2 red). The Gini impurity of the whole dataset is then

$$G_{init} = \sum_{i=1}^3 p(i) * (1 - p(i)) = 3 * \left(\frac{1}{3} * \frac{2}{3}\right) = \frac{2}{3}$$

The two branches have impurities

$$G_{left} = (0 * 1) + (1 * 0) + (0 * 1) = 0$$

$$G_{right} = \frac{3}{8} * \frac{5}{8} + \frac{2}{8} * \frac{6}{8} + \frac{3}{8} * \frac{5}{8} = \frac{21}{32}$$

The Gini Gain is

$$Gain = G_{init} - \frac{1}{9}G_{left} - \frac{8}{9}G_{right} = \frac{2}{3} - \frac{1}{9} * 0 - \frac{8}{9} * \frac{21}{32} = 0.083$$

This is not a particularly good split, because the information gain is very low. However, when trying $x = 1.4$, the green classes are separated from blue and red completely,

$$G_{left} = \frac{3}{6} * \frac{3}{6} + \frac{3}{6} * \frac{3}{6} + 0 * 1 = 0.5$$

$$G_{right} = 0 * 1 + 0 * 1 + 1 * 0 = 0$$

The Gain from this split is

$$Gain = G_{init} - \frac{6}{9}G_{left} - \frac{3}{9}G_{right} = \frac{2}{3} - \frac{1}{3} - 0 = \frac{1}{3}$$

which is a much better split than previously. This split successfully creates a **pure** branch (right), since all samples belong to the same class. The first split is now established, so the same process is carried out for the y variable, keeping in mind that the remaining dataset is the observations to the left of $x = 1.4$.

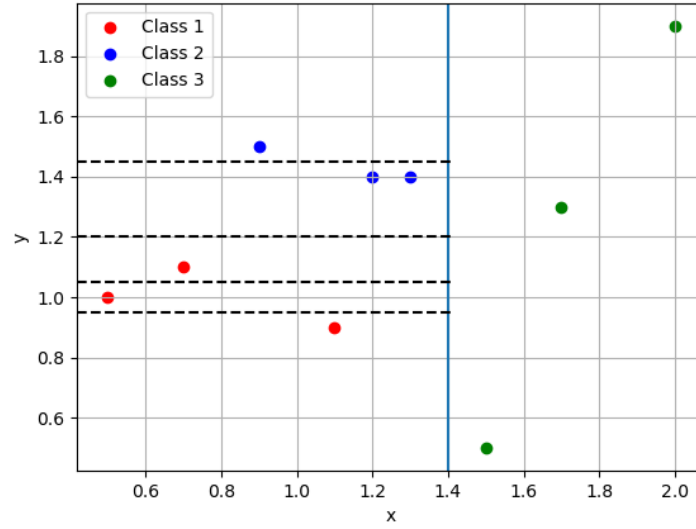


Figure 2.4: Threshold values for y

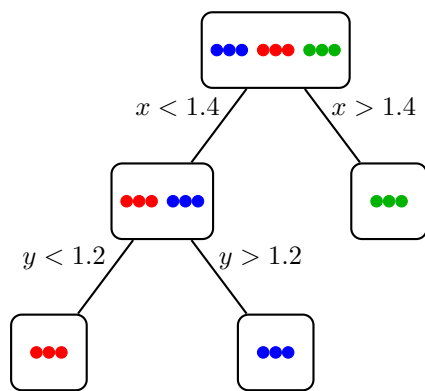
There is only four possible splits here that produce unique branches as shown by Table 2.4:

Table 2.4: Possible splits for the second level

Split	Left branch	Right branch	Gain
$y = 0.95$	•	•• •••	≈ 0.315
$y = 1.05$	••	• •••	0.25
$y = 1.2$	•••	•••	0.5
$y = 1.45$	••• ••	•	0.1

Clearly, the best split is at $y = 1.2$ because it produces two additional pure nodes, resulting in the simple network depicted by Figure 2.5.

Figure 2.5: Resulting decision tree

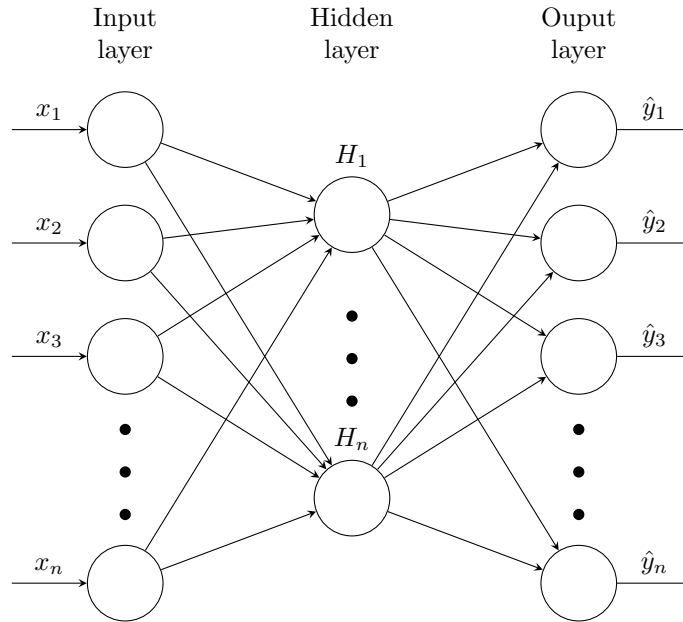


2.3 Neural Network

An artificial neural network (ANN) is based on the biological definition of a neuron, where different layers make up a network which inputs propagate through to be classified by the the last layer. This is a supervised learning algorithm that learns a function $f(\cdot) : R^m \mapsto R^o$ by training [41], where m is the number of features in the input vector and o is the output dimension.

An ANN consists of multiple layers: input, hidden layer(s) and an output. In each hidden layer, there are r neurons that carry out the calculations (note that r might be different for each hidden layer), whereas in the input and output layer there are typically one neuron each ³. The input layer takes in n inputs of x , each with m features, $x = [x_1, x_2, \dots, x_m]$. The hidden layer transforms the values from the previous layer with a weighted linear summation, immediately followed by some non-linear activation function $\sigma(x)$ [22]. Several hidden layers may follow each other immediately, however a maximum of two are usually more than enough for a task [49]. The output layer is the evaluated prediction $\hat{y}$, an $n \times 1$ column vector, which has one output for each vector of x regardless the number of its features. Consider the simple network depicted by Figure 2.3.

Figure 2.6: Simple Neural Network with one hidden layer



³in the case of multi class classification, for k classes there are k output neurons for **each** input $x_i \in x$

2.3.1 Propagation

The way ANNs make sense of the input is by having individual weight vectors W associated with each layer that weight each feature of the input. On the creation of a network, the entries of the weight vectors are by default random numbers on the interval $[0, 1)$. The network needs to be trained to memorise the relationship(s) present in the data in order to make the future predictions. Note that there can be a single bias unit included in the input- and following hidden layers, which allows the fitting of data even if all input values are 0 [44].

When some input x enters the network, the neurons in the current layer carry out a weighted linear summation with their individual weight vector W , described by $x' = x \cdot W$. Note that this is the same as $x' = x_1 * w_1 + x_2 * w_2 + \dots + x_m * w_m$. Then, the activation function, $\sigma(x')$, processes the output of the layer and the value(s) are passed forward to the next layer which repeats this procedure. Therefore individual weight matrices are kept track of by, and are essentially attached to, each layer. There are many neurons that have their own weights for each layer - for r neurons, there are r weight matrices associated with **each** input n . Once the initial input reaches the end of the output layer, one propagation has been completed, at which point it needs to backpropagate to update the weights W relative to the error.

2.3.2 Backpropagation

Let's notate each weight vector, W_{ij} as the link between layer i and j , similarly x'_i as the output of layer i and x_i as the vectors in the layer. Once the first propagation is finished, a loss function - in classification's case, Cross-Entropy [72] - is calculated, which in binary case is

$$Loss(\hat{y}, y, W) = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y}) + \alpha \|W_{ij}\|_2^2$$

where $\alpha \|W_{ij}\|_2^2$ is an L2-regularization [32] (penalty) term. $\alpha > 0$ is a non-negative hyperparameter controlling the magnitude of the penalty [74]. After the loss has been computed, it is propagated starting from the output layer backwards, updating each W on the way. The gradient $\Delta Loss_W^{ij}$ of the loss with respect to the weights W_{ij} is computed and deducted from W_{ij} :

$$W'_{ij} = W_{ij} - \eta \Delta Loss_W^{ij}$$

[38] where $\eta > 0$ is the learning rate. The repeated updates then minimise the loss function, thus converging to the most optimal relationship. The learning rate is defined by the optimizer used, which in this case is simply the Adam [59] optimizer.

2.3.3 Training

Once the network has propagated forwards **and** backwards **once**, it has done **1 epoch** of training. Training worth of d epochs means that the inputs are

propagated through the network, the weights are updated and this is repeated another $d - 1$ times. Then the network is used on a test set to determine its performance according to some metric. By default [74], training is carried out for a specified number of iterations, or alternatively when the improvement in loss is below a certain, small threshold.

2.3.4 Activation function

The activation function's role in the model is to produce a non-linear signal for the next layer [22]. Considering that neurons calculate weighted linear sums, without an activation function each output signal is simply a linear function, which is easy to solve but limited in complexity; it would make the network a linear regression model[101]. There are a handful of non-linear functions $\sigma(x)$ that are commonly used depending on the context of the problem [101]. It is possible - and often required - to use different activation functions layer to layer.

Sigmoid. The logistic function is on the interval $[0, 1)$, described by

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Its prediction $\hat{y} \leq 0.5$ will belong to Class 0 and $\hat{y} > 0.5$ to Class 1. This activation function is standard for binary classification but there are some concerns. The function is not zero centered and has a slow convergence - these properties make the derivative of the function unstable, resulting in gradient updates going too far in either direction during backpropagation [101].

Hyperbolic tan. Given by

$$\sigma(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

, $\tanh(x)$ is on the interval $[-1, 1]$. The function is therefore zero centered, making the overall optimisation easier but still suffers from the vanishing gradient problem [101] whereby weights can die out.

Rectified linear unit (ReLU). ReLU is proven to be more effective than $\tanh$ because it does not suffer from the vanishing gradient problem [61, 108]. It is given by the simple function $\sigma(x) = \max(0, x)$. Whilst there is advice against using it in the output layer [108], some studies have experimented with using ReLU in place of *softmax* [1, 2, 4]. Generally, an advantage (and sometimes a disadvantage) is that this activation function can cause neurons to die where their weights become 0 [108, 90, 22].

Softmax. It is given by

$$\sigma(x) = \frac{e^{x_i}}{\sum_{j=1}^J e^{x_j}}$$

frequently used in multi-class classification problems as the output layer activation function [103, 56, 106] to calculate the probability of x belonging to each class.

ReLU as the default activation function in the hidden layer(s) [22], and *softmax* or *sigmoid* in the output layer [45] is a combination often recommended [90] for multi or binary class networks respectively.

2.4 One-hot encoding

The concept of one-hot is an encoding technique that helps preparing data set for various Machine Learning algorithms, like classifiers and regression models [23]. In the world of mathematics and computer science, one often carries out mappings of text data to numerical, for example let $m = \{Jan, Feb, \dots, Dec\}$ and $M = \{1, 2, \dots, 12\}$ then $f : m \mapsto M$. It is natural to represent chronological data as such, especially when data is stored for later processing, but this can lead to algorithms learning non-existing or biased patterns in the data [68, 19].

Example. Let's label and store days in a single feature, *day*, with its values satisfying $1 \leq day \leq 7$. Feeding this data in with several other columns of data may teach the algorithm that the relationship investigated is much more prevalent on Sundays, but that could be down to the simple fact that 7 is the biggest number in that feature.

Table 2.5: Example rows of data without one-hot encoding

$feature_1$	$feature_2$	...	<i>day</i>
0.5	0.7	...	1
0.1	0.4	...	2
0.7	0.9	...	3

To overcome this, introduce a binary feature for each integer value in the feature, teaching the model to be able to differentiate between days rather than taking the values as natural ordering between items [23]. This means a simple transformation of Table 2.5 to Table 2.6:

Table 2.6: Example rows of data after one-hot encoding

$feature_1$	$feature_2$	...	<i>monday</i>	<i>tuesday</i>	...	<i>sunday</i>
0.5	0.7	...	1	0	...	0
0.1	0.4	...	0	1	...	0
0.7	0.9	...	0	0	...	0

2.5 N-Fold Cross validation

When building a classifier model, not all available data is used for the training, rather the set is divided up into two sets, training and testing. This division

is done by first shuffling the data randomly, then taking a certain proportion (usually 10 or 20%) [86, 92] of all data and delegating it as the test set whilst the rest of the data is trained on. Once the classifier is trained, its metrics are determined by using the model on the delegated test set and determining the proportion it gets right.

Not only will this in itself not give a true metric of the data [46], but the potential of the data set is also being wasted by not all of it being used [46]. Therefore it is desired to use a cross validation technique, which uses the entire set of available data for both training and testing. There are different approaches available like leave-one-out [98], but let's focus on the simple n -fold [7] approach here:

Algorithm 2 N-Fold cross validation

1. Divide the data set S into n even sized partitions
 2. Iterate up to $i = n$
 - 2.1. Take the partition at index $i = 0$ as the test set and remaining $n - 1$ partitions as training
 - 2.2. Train a model on the training set and use the delegated test set to evaluate the model
 - 2.3. Retain model metrics and discard the model
 - 2.4. Repeat for next i
 3. Take an average of the retained metrics to get a true score across the entire available data
 4. Repeat process for a model with different hyperparameters if required
-

Values of n are chosen such that a partition is large enough to be statistically representative of the broader data set. Typical values are $\in 5, 10$ as these usually create large enough partitions. A higher value of n leads to a less biased model [88] (as all data points are being used) and helps avoid overfitting [88], but there might be a higher variation if there are any outliers in the set [99].

Chapter 3

Method

3.1 Overview

The goal of this project is to build a classifier that is able to determine the state, S , of a train crossing at a random time t during a day d . To realize such a classifier, there needs to be a time series data stream by the day, capturing the times of a level crossing barrier descending and ascending, and the length of this interval. The series needs to be broken down by the day, to account for the simple fact that traffic patterns change according to the day of the week as well as the time of the day. Therefore, an overarching classifier must be able to differentiate between days of the week.

The state S can simply be binary labels to indicate only the untraversability of a rail crossing (False when the barrier is ascended, and True for a descended position), or multiple classes that are assigned based on some other feature of the observations. Since the interest is in not only in the current state, but also in **how long the state remains the same for**, classes are assigned to each sample based on the length of the interval. This means that when predicting a state S , what is being predicted is how long the barrier is likely to be down for and not just whether it is currently untraversable or not.

3.2 Data Collection

Possibly the most optimal way to capture a time series on the movement of a level crossing is using a Raspberry Pi, programmed to take and analyse real time video. Due to unforeseen technical difficulties, it was not possible to set the equipment up which led to another, albeit more crude approach of simulating the data series based on time-tabled information. The time series can be artificially created using collected time table information, and manual measurements of how the crossing behaves relative to the traffic.

3.2.1 Web scraping

Web scraping is a technique [67] that allows users to extract specific information from a web page programatically for some purpose. National Rail provides a service [81], which shows the times of departing and arriving trains by station in real time. The service provides information on the delay of the trains and whether they are cancelled. For each train, the following are stored:

- train's origin,
- destination,
- due time (time tabled information),
- delay (in minutes),
- estimated time of arrival (due time - delay),
- whether it is cancelled or not
- the name of the day (e.g. "monday")
- date

as also illustrated by Table 7.1. A script was created in python ¹, to continuously look at arriving and departing trains at Lincoln Central station, saving all into a database for later processing. For the purpose of this project, the decision was made to only collect trains traveling to and from Lincoln Central, because there are other measurements to make manually, which may not generalise to other stations. The data collection was launched on the 1st of December, 2019, and was ran for 13 weeks, saving a total of 12172 ² distinct trains by the end of this period - therefore resulting 13 sets of observations belonging to each day across the short period.

3.2.2 Manual data collection

In addition to web scraped data, it was essential to learn about the timing element of the barriers, so that some rules can be coded in the simulation algorithm about the general behaviour of a crossing. The measurements were fairly intuitive, because there was no element of time offset due to travel speed at this station - for most crossings, one must consider how long the train takes to get near to the barrier, relative to the station timetable. Manual readings were carried out in person for the following aspects:

1. seconds elapsed between barrier initially descending and arrival of first train in session
2. seconds elapsed between two consecutive trains passing

¹*scraper.py* file in the repository [34]

²saved in the *trains.db* file [34]

3. seconds elapsed between last train passing and barrier ascending

When carrying out the measurements, the stopwatch was always started and stopped when the train was in line with the points A and B established next to the crossing, to ensure that readings and their accuracy are the least affected by human error. When measuring the barrier times, the stopwatch was started and stopped **on the first movement of the barrier**, rather than the full time it takes to shift into the given state.

The data acquired on the trains via webscraping is used as a time-tabled ground information, which in conjunction with the aforementioned manual measurements can be used to approximate the movement of the barriers relative to the given train departure and arrival times. This simulates the data stream that a sensor, or a camera setup would have gathered. The manual measurements are found in Table 7.2, Table 7.3 and Table 7.4, respectively to the list. The algorithm using this data is discussed in-depth in Section 3.3.

The integrity of the manually collected data remains questionable however, as these timings were all collected on random days, at random times of the day. This means that intervals measured in the evening might not be representable of the early morning and vice versa. Based on the behaviour of the level crossing, the barriers are controlled by a computer depending on some internal constraints (and safety measures), meaning that given enough observations the behaviour is reproducible.

3.3 Simulation

In the discrete dataset, there needs to be information about when the barrier initially descended and ascended, the length of this interval and the number of trains that passed during this time. The times in Table 7.2 are fairly random with $\sigma = 30.01$, $\mu = 52.29$, therefore simply generating random numbers for this variable will be sufficient. Plotting the numbers will allow to see which distribution the collected data is most similar to. Secondly, Table 7.3 outlines the time waited between two trains passing **without the barrier changing state** - this means that given enough measurements, one should be able to find the maximum time in seconds that the crossing waits for. Due to the limited number of measurements, simply the highest value is taken. Then, Table 7.4 outlines the time waited before the barrier ascends when all trains have passed; for this set $\sigma = 5.25$, $\mu = 8.75$ so simply use μ as the waiting time before changing state.

To find the characterising distribution of Table 7.2 values, let's consider its kernel density estimate depicted by Figure 3.1:

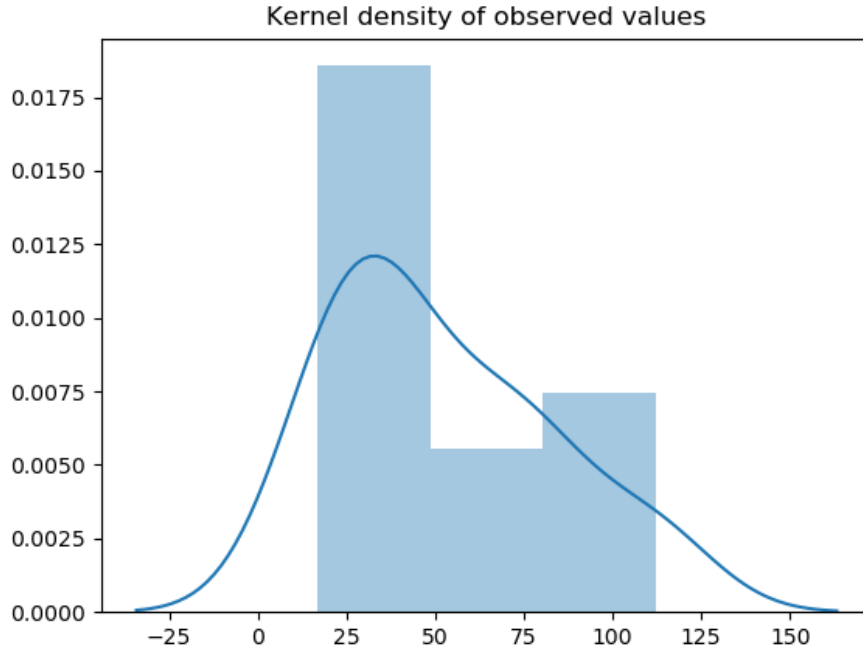


Figure 3.1: Kernel density of observed values of time difference between barrier descension and first train passing (in seconds)

A sufficing distribution needs to be on the interval of $[0, 1]$ and have shape

parameters to characterise the almost straight line after the peak. One fitting the criteria is the Beta distribution [105, 70], which with the values of $\alpha = 2.5$ and $\beta = 6$ creates a close estimate of Fig 3.1. Any values drawn from this distribution are scaled back up onto the interval of [17, 112].

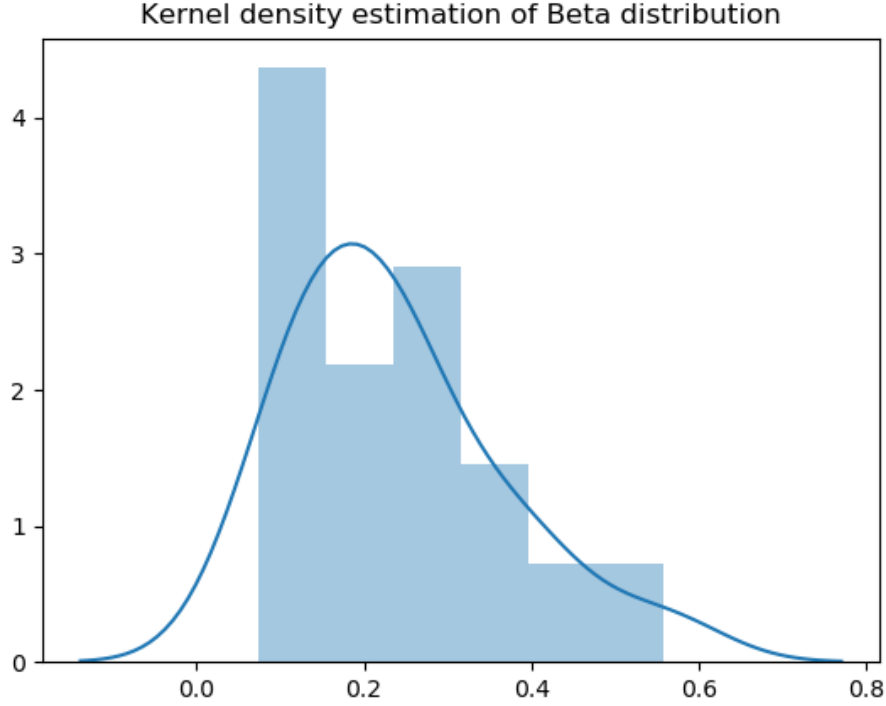


Figure 3.2: Kernel density estimate of $Beta(2.5, 6)$

Let $X \sim Beta(2.5, 6)$, and $\uparrow X$ be the rescaled and rounded random number onto the interval [17, 112]:

$$\uparrow X = X(112 - 17) + 17$$

[54]. This shall be used from hereon out when a random number is generated for this variable.

Let t_d be the time between barrier descending and train passing, $\uparrow X$. Let t_w be the max waiting time between two trains passing, $max(Table7.3)$ and t_u be the elapsed time between last train passing and the barrier ascending, $\frac{1}{n} \sum_{i=0} Table7.4_i$.

Further define S to be the set of all entries and S_d the set of trains on date d where $S_d \in S$. The data points, t_i are objects with attributes outlined in

Table 7.1, such that $t_i \in S_d$. Denote an object's attribute as $t_i.x$ where x is any attribute of the object.

Having fixed the notation, the algorithm ³ of artificially simulating the data stream for the barrier's movement, relative to arriving and departing trains, is as follows:

Algorithm 3 Data stream simulation

1. Get all trains for the first date d , call this set S_d .
 2. Clean S_d by getting rid of cancelled trains, duplicate entries and obvious outliers.
 3. Sort trains ascending according to their estimated arrival (includes delay of train).
 4. Iterate through trains t_i :
 - 4.1. Let the time of barrier being down $b_d = t_i.est - t_d$
 - 4.2. If $t_{i+1}.est \leq t_i.est + t_w$ then wait for next train. Keep iterating each sequential train, referring to its index as x **relative to** i , until for some x , $t_{i+x}.est > t_{i+x-1}.est + t_w$. Ignore trains t_{i+1} to t_{i+x} once iteration proceeds because they have been processed as part of this step.
 - 4.3. Barrier up time, b_u , considering x is the number of **extra** trains passing, is $t_{i+x+1} + t_u + \delta$ where δ is any extra accumulated time (long train, freight time pass time, etc.)
 - 4.4. Make entry of $b_d, b_u, b_u - b_d, x$ in file for this date d
 - 4.5. Proceed to next **unprocessed** data point.
 5. Date d processed, start from step 1 again with $d = d + 1$ (if exists).
-

Algorithm 3 aims to generate an accurate data stream of how a crossing barrier would behave relative to the rail traffic. The waiting times (both for the movement of the barrier and in between trains) are already known by manual measurements, so it is possible to check if the following trains ⁴ are within the same barrier session or not. This is simple by adding the t_w constant to the estimated arrival time of the current train; if the resulting quantity is more than the estimated arrival of the next train, the next train would in life follow shortly and thus are in the same session. The same logic is applied on following trains, until one does not satisfy this condition. When the session is over, the waiting time of barrier ascension is added to the last train's estimated arrival time and

³is implemented by *Simulation.py* script, found in the repository [34]

⁴to the currently processed one

then it is all stored in an entry of a file.

3.4 Data Preparation

To reiterate results from the previous section, the structure after Algorithm 3 finishes is as follows:

Table 3.1: Structure of csv files

b_d	b_u	$b_u - b_d$	x
08:45:42	08:47:15	93	1
09:58:24	09:59:15	51	1
10:34:52	10:36:15	83	1
$\vdots$	$\vdots$	$\vdots$	$\vdots$

There is a .csv file for each processed date d , all stored by the day of the week, which are later collated to produce an overarching data set. There are two major problems with the structure of the data in its current form:

1. Once collated, needs an extra column to signify the day
2. The timestamps can not be processed as a *datetime* structure

3.4.1 One-hot

To address Problem (1), one is inclined to simply include a column, *day*, which maps numbers 1 to 7 to the days of the week (excluding 0 because it can lead to algorithms ignoring rows of data on Mondays). However, this would actually increase the weighting of any correlation found towards the end of the week. As outlined in Section 2.4, there needs to be a binary column for each value of the hypothetical *day* column to make it clear to the classifiers that those are different *items*.

3.4.2 Dealing with timestamps

Problem (2) however is much trickier to deal with. The main and foremost problem is that it is not possible to process the data set because there are columns of *datetime* objects, but algorithms can, natively, only deal with *number* data types. It is possible to carry out a simple transformation, converting the timestamps into seconds (from midnight), but that in itself is not sufficient. The model would not be particularly good, because it would not know that the data is cyclic. Assume, that there is routinely a train 2 minutes after midnight. This means that the barrier will more than likely drop just before midnight (let's say 23:59) and ascend sometime after midnight (00:05). Clearly the difference in time is 360 seconds, but in this form they will appear to be 23 hours and 54

minutes apart, and the connection will never be made [64].

To overcome this, the hypothetical seconds columns, b_{td} and b_{ta} , are replaced with a combination of $\sin$ and $\cos$ transforms. The symmetry of these functions means that if only one is used, each point would touch the graph in two places, making them indistinguishable from one another at times - hence the need for the combination of both [64, 55] which uniquely maps every second to a combination. The process is carried out for both b_d and b_a , resulting in a pair of equations each **instead** of the b_d and b_a columns. Let θ be the total seconds in a day, then for each data row i , two pairs of features are created and b_d , b_a are dropped.

$$\begin{aligned} \sin_d &= \sin(2\pi \frac{b_{td}}{\theta}), & \sin_a &= \sin(2\pi \frac{b_{ta}}{\theta}), \\ \cos_d &= \cos(2\pi \frac{b_{td}}{\theta}), & \cos_a &= \cos(2\pi \frac{b_{ta}}{\theta}). \end{aligned}$$

Consider the following figure which represents one day's worth of descension times:

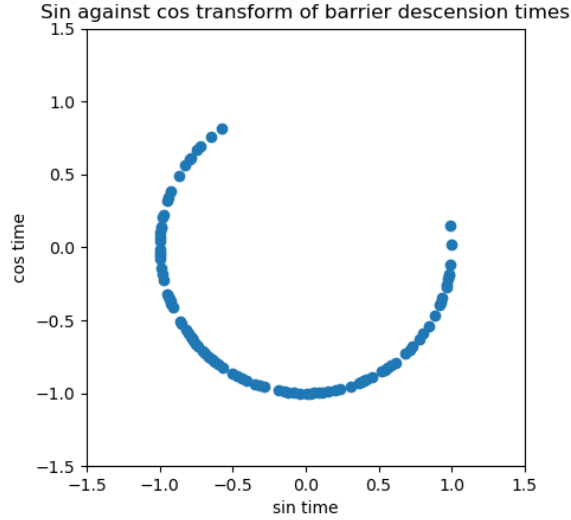


Figure 3.3: Visualisation of the $\sin(x)$ and $\cos(x)$ transform couple

Figure 3.3 clearly shows that the descension times of the barrier are represented on a 24-hour clock, so now the distance between two points is exactly the difference in time as expected.

3.4.3 Feature engineering

Feature engineering is the concept of inventing new features from the existing data set or changing the way they are presented [16], which plays a major role in a model's success or failure [37]. There are well defined methods available

for this purpose [12], some of them being as simple as breaking up the date into three features (day, month, year), or including a feature for whether the entry is during business hours, or what the business quarter or season of the year [18] it is. The process of this is not always easy because it is an iterative process [37, 109], requiring continuous experimentation of the designed features [109]. In terms of a time series, a classic engineering technique is decomposing the signal into **season**, **trend** and **stochastic** components as given by STL decomposition [30]. Bearing in mind that a time series usually measures the magnitude of something over a long time (e.g. number of flights, tonnes of sold goods, etc), this technique can not be applied here, but it is a motivation for deriving features. Before deciding to add any new information, let's consider binary against multi class classification for this problem.

3.4.4 Binary vs multi class classification

A binary classifier would be able to predict the status of a crossing at $f(t)$ but it would have no concept of how long it is likely to remain as such. A typical binary classifier could be created with the following features:

Table 3.2: Binary classifier feature set

$\sin_d$	$\cos_d$	state	mon	...	sun
----------	----------	--------------	------------	------------	------------

$\sin_a$ and $\cos_a$ were dropped because for new observations, the time of ascension is unavailable and thus the classifier could not be used. Here *state* is a boolean describing whether the crossing barrier is **untraversable** (when barrier is down, state is *true*). The model described by Table 3.2 can be implemented as is, or its performance could be boosted with some of the following methods in this section ⁵. Instead, consider that this required dropping some features which is wasteful. To be able to predict the likely time a crossing will be untraversable for, the $\sin_a$, $\cos_a$ and *length* features are reintroduced to the model and transformed later.

Since the prediction needs to entail both the state and the predicted length of a sample, there must be more than 2 classes and thus the *state* feature needs to be one hot encoded, where there is one binary column for each possible classification label. The labels can be assigned according to the length of time the state remains the same for in each data row. The length of interval against the descension time is plotted to see the distribution of the interval times:

⁵in this case only three additional features were used for the binary model, a time lag of 1 of $\sin_a, \cos_a$ and *state*

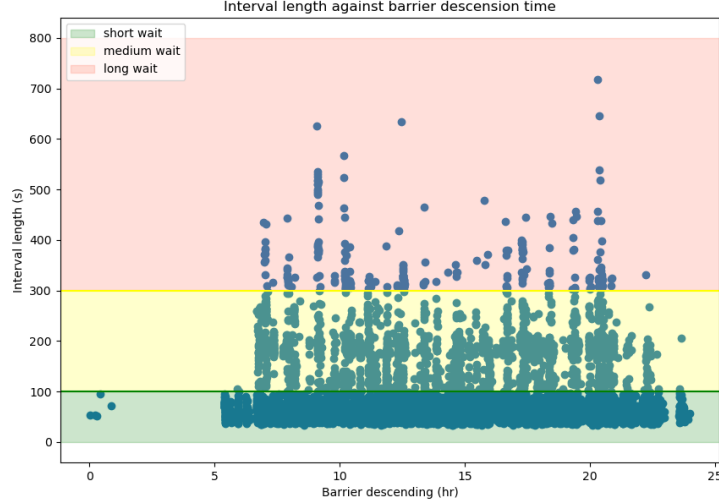


Figure 3.4: Visualisation of the data set and class distribution

There are clear partings in the graph, at $y = 100$ and not so obviously at $y = 300$, which makes 3 distinct classification labels and an extra one for the original *False* value. There will be a *none* label showing the barrier is ascended; a **short** ($0 < l \leq 100$), **medium** ($100 < l \leq 300$) and **long** ($l > 300$) classification to characterise the approximate waiting time. Let these be c_1 to c_4 respectively. The structure is then as indicated by Table 3.3.

Table 3.3: Initial multi class feature set

sin_d	cos_d	sin_a	cos_a	length	c_1	c_2	c_3	c_4	mon	...	sun
---------	---------	---------	---------	---------------	-------	-------	-------	-------	------------	------------	------------

Features $\{c_1, c_2, c_3, c_4\}$ are the one hot encoded binary features, y , being predicted.

3.4.5 Time lag

Considering a real life application, features $length$, sin_a and cos_a can only be used to predict $f(t+1)$ but not $f(t)$, because the information is not yet available at the time of the query q . Therefore these values can be **shifted**, so that the next rows use these values.

A shift is also described as a lag - they are the classical way time series forecasting problems are transformed into supervised learning problems [18]. This introduces the concept of short term memory ⁶into the model since the prediction of t is determined using information from the data point $t - 1$. It can

⁶mimicking an LSTM Neural Network [94, 110, 52]

be extended to as many time steps ahead as needed, but in the case of this problem only a time lag of 1 is chosen. Lags of more than 1 may or may not impact the performance - the need for a time lag of more than 1 is dependent on the correlation a given observation has with the past values [85]. A lag of k means that the first k items have to be thrown away because the data will be incomplete in those rows. Let $\Downarrow_k f$ denote the time lag of k for a feature f .

Example. Consider the subset of three items from the data set after a time lag of 1 applied on $\sin_d$ and $\cos_d$:

Table 3.4: Shift example

$\sin_d$	$\cos_d$	mon	...	sun	$\Downarrow_1 \sin_d$	$\Downarrow_1 \cos_d$
0.851117	-0.524977	1	...	0	NaN	NaN
0.831470	-0.555570	1	...	0	0.851117	-0.524977
0.811574	-0.584250	1	...	0	0.831470	-0.555570

The incomplete table Table 3.4 showcases the form of the data after a time shift of 1 on one given feature.

Following on from Table 3.3, carry out $\Downarrow_1$ on $\sin_a$, $\cos_a$ and $|length|$ where $|length|$ is the normalised set of lengths where items are $z_i \in length$ and $\hat{z}_i \in |length|$:

$$\hat{z}_i = \frac{z_i - \min(length)}{\max(length) - \min(length)}$$

The derived features therefore are indicated in Table 3.5 where labels in bold are the classification feature(s).⁷

Table 3.5: Resulting set of features used for classification

$\sin_d$	$\cos_d$	$\Downarrow_1 \sin_a$	$\Downarrow_1 \cos_a$	$\Downarrow_1 length $	mon	...	sun	C₁	C₂	C₃	C₄
----------	----------	-----------------------	-----------------------	-------------------------	-----	-----	-----	----------------------	----------------------	----------------------	----------------------

⁷The used data is available to view in the repository [34]

Chapter 4

Results

4.1 Classification process

The prepared data set is now ready to be used by a supervised learning algorithm. Although Chapter 2 gives away the classifier candidates, the decision for their use are as follows. Since $\sin_d, \cos_d$ and $\sin_a, \cos_a$ are values on the interval $[-1, 1]$ and form a 24-hour clock, Nearest Neighbour might be the most intuitive classifier to experiment with, since entries are adjacent in the context of the clock format. Whilst neural networks are the most data sensitive classifier, it is worth assessing its performance, because with correct feature engineering and a sensible structure it has the potential to do well. In addition to this, Random Forest is a robust and versatile classifier, which typically excels in most problems.

The range of classifiers are cross validated with the same number of n folds, and are all compared at the end. For all the classifiers defined below folds $n = 5$, translating into a 80/20 split for training and testing respectively. The class distribution is depicted by Table 4.1. After the discussion of the top performing models for each classifier, the overall best performing model is chosen to see how they compare to one another.

Table 4.1: Number of samples by class

Class	# samples
none (c_1)	8729
short (c_2)	6724
medium (c_3)	1681
long (c_4)	235
Total	17369

4.1.1 Performance assessment

Before diving in the results, it is essential to understand the metrics according to which the models are examined. Table 4.1 shows that this data set is quite unbalanced - observations of c_4 are **rare** cases, which means that a model might perform poorly on this class according to some metric. It is therefore vital to understand the metrics available to be able to assess models according to various different aspects.

Accuracy, $acc = \frac{TP+TN}{Total}$, is the most natural but also misleading metric - it is misleading when the set is highly unbalanced [48, 53, 27]. Let's consider an example of a total of 100 entries, 10 out of which belong to Class A and 90 belonging to Class B. In this case if the classifier guesses Class B for each entry, it will achieve a 90% accuracy purely because of the distribution of examples.

Precision is the fraction of relevant instances among the classified instances, or otherwise, the number of correctly identified samples within the given class, given by $Pr = \frac{TP}{TP+FP}$ [53]. This metric becomes important when the model in question should absolutely make correct classifications [77]. In the case of this data set, the goal is to have a high value of precision for across all classes. This is important because it means that the classifier can reliably classify entries and that when it classifies an entry, the classification is correct.

Recall describes the number of correctly identified instances over the class size, or otherwise, $Rc = \frac{TP}{TP+FN}$ [53]. Let's consider a simple example of a picture containing 15 dogs and some cats. Suppose a script identifies 9 dogs in this picture. Out of the identified 9 dogs, only 7 are actually dogs, while the remaining 2 are cats. The precision then is $\frac{7}{9}$ and recall is $\frac{7}{15}$. Therefore a good model should have both a high precision and recall rate over high accuracy.

F1-Score is the harmonic mean of precision and recall rate [77], $F1 = 2 \frac{Pr * Rc}{Pr + Rc}$, insensitive to the distribution of the incorrect predictions over the classes [42]. The harmonic mean is a truer average than arithmetic or geometric average because it is always the least of the three [107]. It's best value is at 1 which would mean a perfect precision and recall rate.

For each of these metrics, there is a **macro** and **micro** metric available [93], which is mostly significant for classification problems with classes $c > 2$. In general, the **macro** version will compute the metric independently for each class and then take the average. **Micro**, on the other hand, computes a weighted average [8]. Micro averages are important for unbalanced data sets because the class imbalance is captured and thus the average performance remains high, even if there are no classifications of one class [80]. The models in this case are evaluated according to the **macro average** and **weighted average** of the f1-score.

4.1.2 Reducing sample size

Each model will be trained using the available data set, as well as a reduced data set where classes are more balanced. Let $|c|$ be the number of samples belonging to a class label c . As by Table 4.1, $|c_4| = 235$ and $|c_3| = 1681$. If all classes were reduced to 235 samples, it means 47 samples per class per fold, which is not a large enough data set [20], therefore $|c_1|$ and $|c_2|$ need to be reduced to the second smallest set, such that $|c_1| \approx |c_2| \approx |c_3|$. Graphing how the metrics change for a model depending on the sample size can help identify how to downsample the data set by brute force, whilst maximising the weighted performance. For the next figure, $|c_1| \approx |c_2| \approx |c_3|$ as long as the number of samples < 1681 , after which point $|c_3| = 1681$ and only $|c_1|$ and $|c_2|$ are increased.

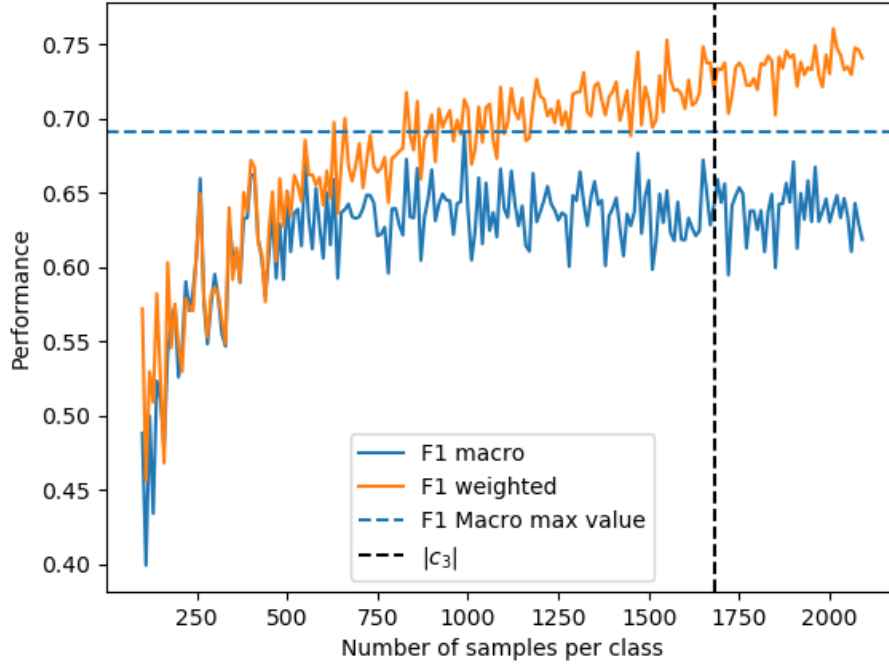


Figure 4.1: Finding the optimal sample size for RF

The value of the macro score peaks at $n = 1000$, showing that Random Forest actually benefits from downsampling c_3 further¹. The weighted score is continuously increasing with the increasing sample size, because as more samples are reintroduced into the classifier, the dense classes c_1 and c_2 will have a higher performance, since more are correctly recognised within those classes. It will

¹and other classes with it

also perform worse and worse on the rarer cases c_3 and c_4 because they are being outweighed with the increasing number of samples in the other classes, meaning that the macro average performance stays in the same region and the weighted averages keeps growing. This also shows that for k classes C where $k - 1$ are balanced, the coefficient α in the relationship $\alpha|C_k| = |C_{k-1}|$ defines the maximum multiple the size of the balanced classes must be in relation to the rare case class. The macro metric does not grow if the last class starts becoming disproportionate. For Random Forest, $\alpha = \frac{1000}{235} \approx 4.26$ and so let $|c_2| = |c_3| = |c_4| \approx 1000$ for the reduced model of Random Forest.

Nearest Neighbour on the other hand actually benefits more from having additional samples rather than just balance; the same general relationship is discovered as before, except the peak is reached at $n = 1800$. However, this means that $|c_3| = 1681$ and so it is slightly out of balance with c_1 and c_2 .

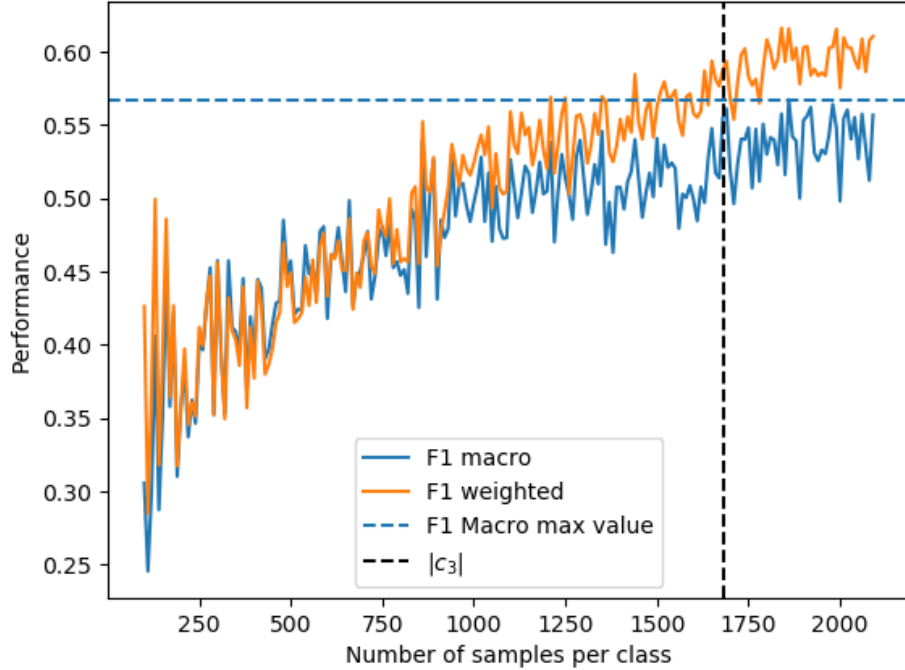


Figure 4.2: Finding the optimal sample size for NN

Therefore for the Nearest Neighbour model let the reduced data set such that $|c_1| = |c_2| = 1800$ whilst keeping all samples in other classes.

This is a bit more difficult to figure out on an artificial neural network, because each model needs to be trained a lot of epochs and so the training time

can quickly get out of hand. Having trained each model until convergence the graph is as follows:

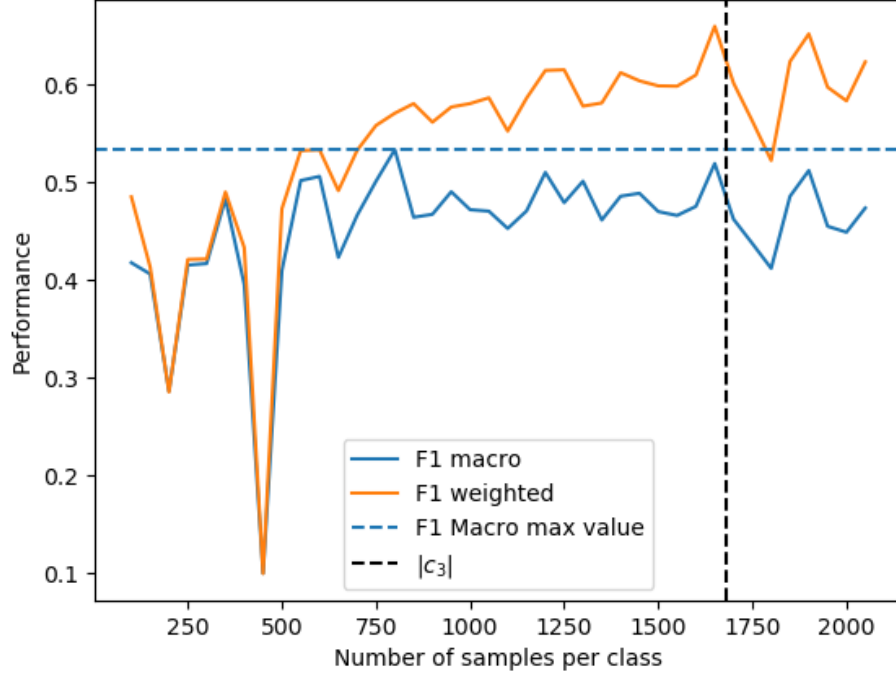


Figure 4.3: Finding the optimal sample size for ANN

In this case, the relationship is not very obvious due to the bigger step size of n , however the two metrics roughly follow the same trend. Considering that ANN is possibly the most data sensitive classifier, the decision was made that for the reduced data set, c_4 and c_3 will stay as is originally, whilst c_1 and c_2 are down sampled to 1700 observations, which is where the second biggest peak is for the macro score.

4.1.3 Nearest Neighbour

Defining the hyperparameters for Nearest Neighbour is fairly easy because there are not many available; the chosen ones were $n - neighbours$ and the p value. The range of values selected for these features are $n - neighbours \in \{3, 5, \dots, 61\}$, $p \in \{1, 2\}$ where 1 and 2 map to Manhattan distance and Euclidean distance respectively [73].

After completing the first round of cross validation, it was noted that p values have very little effect on the performance (less than ± 0.0001 change in score), therefore they are excluded here. The top three candidates are then as follows:

Table 4.2: Nearest Neighbour Classifier Cross Validation top candidates

micro f1-score (%)	$\pm$ across groups (%)	n-neighbours
78.04	0.8444	5
77.81	0.90954	3
77.81	1.06127	7

The top candidate is trained again without any folding to see the *precision*, *recall* and *accuracy* values:

Table 4.3: Nearest Neighbour Classifier

	precision	recall	f1-score	samples
up	0.87	0.91	0.89	1742
short-wait	0.80	0.75	0.78	1342
medium-wait	0.44	0.33	0.38	337
long-wait	0.36	0.17	0.23	53
micro avg	0.81	0.78	0.79	3474
macro avg	0.62	0.54	0.57	3474
weighted avg	0.79	0.78	0.79	3474
samples avg	0.78	0.78	0.78	3474

The performance of this classifier is surprisingly good, even with such an unbalanced data set. It is clear that there was not enough data for c_3 and c_4 because for classes with more data the f1-score is good (≥ 0.78), but for the others it is poor. Reducing the number of samples belonging to c_1 and c_2 is expected to bring down the performance of those classes but increase the c_3 and c_4 performance:

Table 4.4: Nearest Neighbour Classifier with reduced data set

	precision	recall	f1-score	samples
up	0.71	0.61	0.66	350
short-wait	0.61	0.55	0.57	369
medium-wait	0.60	0.59	0.59	325
long-wait	0.34	0.22	0.27	60
micro avg	0.63	0.57	0.60	1104
macro avg	0.57	0.49	0.52	1104
weighted avg	0.63	0.57	0.59	1104
samples avg	0.57	0.57	0.57	1104

Whilst the performance of classifying c_3 and c_4 examples slightly increased, the

drop in overall performance stems from that with the reduced data set, the classifier is having trouble making reliable predictions for c_1 , c_2 . This clearly showcases that as classes become more balanced, the performance can increase, but it is vital to have a good volume of data as well.

4.1.4 Random Forest

In Random Forest there is a plethora of available hyperparameters [75], the most impactful of them being n-estimators, min-samples-leaf, max-depth and min-samples-split [60, 100, 43]. In this classifier, the criterion (Gini vs Entropy) and the number of features considered at each split proved meaningless in terms of overall performance, which alleviated some training time in the long run - simply the default values were used [75]. Let's consider the range of values to input for each remaining hyperparameter.

n-estimators - the number of trees should be as many as possible, while also considering that this attribute is computationally expensive especially in cross validation. As a result of this, the goal is to keep this number low but representative, so the values defined $\in \{100, 200, 500, 1000, 2000\}$.

min-samples-leaf - there are not many samples in the data set, so this value should be kept as low as possible; it should also be considered that the classifier might not be able to create enough impure sets if the value is too high, so the trial values are $\in \{1, 2, 3, 4, 8, 12, 16\}$, which will allow to see how the performance is impacted with this attribute.

max-depth - this value should not be too large to avoid overfitting [36], the range of values are $\in \{None, 20, 40, 60, 80, 100\}$.

min-samples-split - it is essential to remember that the sample size is not large, therefore the classifier might lose out on some valuable splits if this is set too high; in this case it is worth looking at a larger range of values and see how they compare. The values are defined to be $\in \{2, 3, 4, 6, 8, 10, 12, 16\}$.

By the end of training, each combination of hyperparameters are evaluated exhaustively on each partition of the folded data set, therefore the list of results sorted by the performance metric in descending order is the list of best to worst models. Here are the top 5 candidates:

Table 4.5: Random Forest Cross Validation top candidates

micro f1-score (%)	$\pm$ (%)	n	samples-split	samples-leaf	depth
84.51	1.019002	500	2	2	80
84.46	1.0179	2000	3	2	None
84.46	0.958	1500	4	1	50
84.45	1.0879	2000	2	2	None
84.44	1.0497	1500	4	2	80

All of these models are fairly similar in terms of their f1-score and so it is difficult to decide the best model based on that alone. Considering the depth, larger values could mean that the model is overfitted; in this case, there is not much difference between the first and second models' performance with depth values of 80 and *None*, which can either mean that **both** or **neither** of them are overfitted. Since the third model is also very close in terms of performance with a depth of 50, it is evident that the models are not being overfitted and so the first model is truly the best model. Rebuilding the first classifier with the outlined properties yields the following:

Table 4.6: Random Forest Classifier

	precision	recall	f1-score	samples
up	0.94	0.91	0.93	1742
short-wait	0.82	0.84	0.83	1342
medium-wait	0.51	0.28	0.37	337
long-wait	0.44	0.13	0.20	53
micro avg	0.87	0.81	0.84	3474
macro avg	0.68	0.54	0.58	3474
weighted avg	0.85	0.81	0.82	3474
samples avg	0.81	0.81	0.81	3474

This model does really well on c_1 and c_2 classes, with respective scores of 93% and 83%, whilst performing better on c_3 and c_4 than Nearest Neighbour did. Interestingly, the precision for c_3 and c_4 are in the region of 50%, but the f1-scores are heavily impacted by the poor recall scores. Balancing the data may improve this, with reduced samples in c_1 and c_2 , as described in Subsection 4.1.2, the performance is as follows:

Table 4.7: Random Forest Classifier with reduced data set

	precision	recall	f1-score	samples
up	0.94	0.81	0.87	188
short-wait	0.78	0.64	0.70	210
medium-wait	0.69	0.61	0.65	195
long-wait	0.71	0.28	0.40	54
micro avg	0.80	0.65	0.72	647
macro avg	0.78	0.58	0.65	647
weighted avg	0.79	0.65	0.72	647
samples avg	0.65	0.65	0.65	647

In the case of this classifier, the macro average of f1 metric has actually **increased** overall, despite a slight drop in performance for c_1 and c_2 . This demon-

strates that a balanced data set can do well even with only a small volume of data and in this case, one might consider using the model Table 4.7 describes, instead of the original highly unbalanced set.

4.1.5 Neural Network

In the case of a Neural Network, the Keras [29] deep-learning library was used to train and test the networks. Each layer must be defined with a number of neurons and an activation function.

My network consists of three layers, the input layer L_i , output layer L_o with a remaining hidden layer L_h . For simpler problems like this, one hidden layer is sufficient [49] to carry out the work. The input layer neurons N_i is defined to be 500, with 4 neurons in the output layer, N_o , equaling to the number of classes. Then defined N_h according to the relation [51]

$$N_h = \frac{|S|}{(\alpha(N_i + N_o))}$$

with $\alpha = 7.5$ and number of samples $|S| = 20000$. In the input and hidden layer a ReLU activation function is used, with a softmax function in the output layer to end up with a probability of the sample belonging to each class. The model uses an Adam optimizer with a learning rate $n = 5 * 10^{-6}$ and a loss function of categorical cross-entropy.

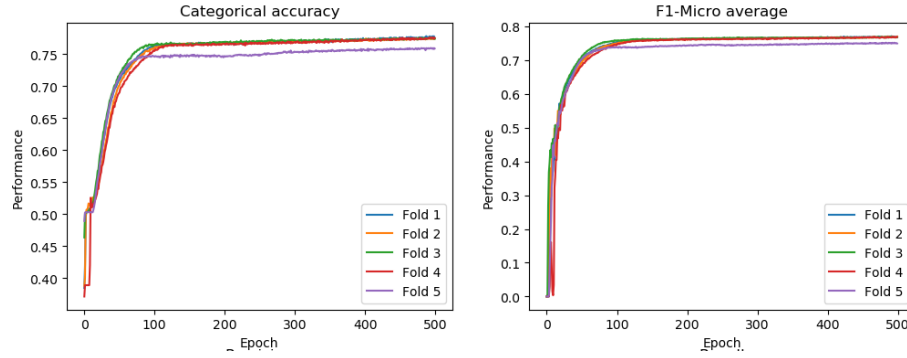


Figure 4.4: 5-fold cross validation of data set

Figure 4.4 shows that each fold trains at the same rate and also almost to the same success. The scoring breakdown per class for this model is as follows:

Table 4.8: Neural Network Classifier				
	precision	recall	f1-score	samples
up	0.89	0.82	0.85	1786
short-wait	0.66	0.91	0.76	1324
medium-wait	0.00	0.00	0.00	332
long-wait	0.00	0.00	0.00	32
accuracy			0.77	3474
macro avg	0.39	0.43	0.40	3474
weighted avg	0.71	0.77	0.73	3474

There are zero-values for two of the four classes and so the macro average is dragged down by these values, but if the classifier is evaluated by **micro** metrics, then 77% is impressive - it shows that if not for the zero-values, the model in general is sufficient. The zero values are likely to have happened because the number of samples in $c_1, c_2 \gg c_3, c_4$, therefore reducing the sample size of the dense classes is likely to increase the performance.

Table 4.9: Neural Network Classifier with reduced data set				
	precision	recall	f1-score	samples
up	0.72	0.88	0.79	336
short-wait	0.47	0.48	0.47	325
medium-wait	0.52	0.47	0.49	358
long-wait	0.00	0.00	0.00	45
accuracy			0.58	1064
macro avg	0.43	0.46	0.44	1064
weighted avg	0.54	0.58	0.56	1064

Although there is an improvement of 4% in the macro average, there is also a substantial 19% decrease in the micro f1-score as c_1 and c_2 samples are removed. There are still no correct classifications of c_4 samples, but as the samples become more balanced, the predictions also balance out across each class. It is demonstrated by the cross validation process in Figure 4.5 for the reduced data set that the models on average perform worse per fold and that the network learns much slower.

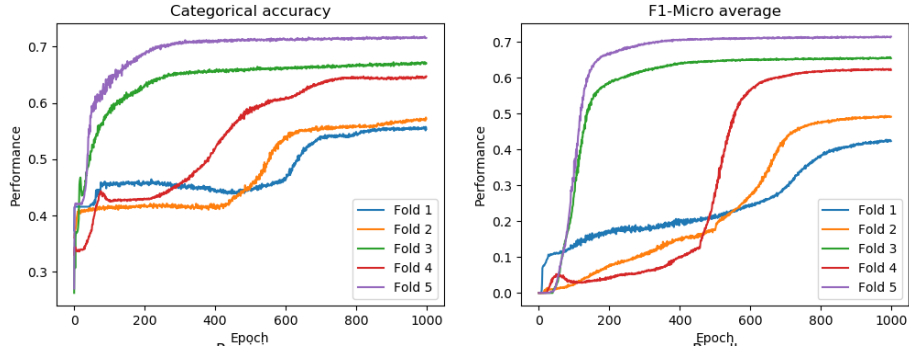


Figure 4.5: 5-fold cross validation of reduced data set

This indicates that whilst the data quality is there, the volume of data is generally not enough. There are 13 weeks worth of data collected, and there are a total of 235 observations for the class c_4 - this is on average 18 observations per week or 2.5 a day.

4.2 Discussion

4.2.1 Best model

The built models are listed according to their **macro average** of f1 score. The highest value signals a model which handled classifications across all classes the best, which is important for the context of the problem because it is important that rare cases are correctly identified.

Table 4.10: Ranking of built models

Rank	Classifier	F1 Score					
		c_1	c_2	c_3	c_4	macro avg	weighted avg
1	RF (Reduced)	0.87	0.70	0.65	0.40	0.65	0.72
2	RF	0.93	0.83	0.37	0.20	0.58	0.82
3	NN	0.89	0.78	0.38	0.23	0.57	0.79
4	NN (Reduced)	0.66	0.57	0.59	0.27	0.52	0.59
5	ANN (Reduced)	0.79	0.47	0.49	0.00	0.44	0.56
6	ANN	0.85	0.76	0.00	0.00	0.40	0.73

As shown by Table 4.10, Random Forest produces the best model according to the macro average when the sample size is balanced between c_1 , c_2 and c_3 . The normal Random Forest model produces the second best results (or the best, according to the weighted average metric), with significant differences in f1 score

of c_3 and c_4 ; whilst the drop in the performance of c_1 and c_2 are not critical. This proves that for classifiers with a rare classification, the balance of data is very important and with only one class being out of balance with the other $c-1$ classes, the model can still do well. The individual scores for c_1 and c_2 are fairly high in classifier (2) and thus it is implied that with balanced c_1 , c_2 and c_3 but more data across all sets, a successful classifier can be built to purpose.

It was expected that a Neural Network will not do too well because of the poor sample distribution and the general low volume of data available. In terms of the scoring, the model was heavily impacted by the fact that it was not able to classify c_3 and c_4 classes at all. Since the rare cases are just as important (if not more) as the normal cases, this classifier does not work well for the problem in this state.

Random forest outperforms Nearest Neighbour but only by 1%, showing a similar performance trend as demonstrated by Noi and Kappas [96]. In general, both RF and NN performed well, however it is indicated by the results of the reduced models that with a balanced dataset consisting of more observations per class would allow the models to achieve much greater metrics.

Since the initial proposal was that the produced model is implemented into a real-life application for real time predictions, the speed of the classifiers need to be taken into account as well to decide the most suitable model. Whilst Nearest Neighbour is the second-best algorithm performance wise, it is slower than both Random Forest and Neural Networks [11] and therefore is less suitable for serving a lot of users. With an increased volume of data, Nearest Neighbour loses even more so performance-wise [35].

This concludes that for cases when the rare cases are important, the reduced RF model is the best to implement in a live system; if rare cases are not as significant to recognise, the non-reduced model performs better on the dense classes c_1 and c_2 . Since here it is important that all classes are recognised reliably, the best model is **RF built using the reduced data set** - it only loses 6% and 13% in c_1 and c_2 compared to the normal RF, whilst gaining 28% and 20% for c_3 and c_4 respectively; the performance balance across all classes is much better hence being more reliable overall.

4.2.2 Methods of improvement

Whilst a well performing classifier is achievable using the developed method, there are some ways the results could be improved, but obviously the biggest improvement could be done by gathering more data.

Longer collection time. The data collection ran for 13 weeks, starting December 2019 until March 2020, so running for longer could have improved the results. However, this was nearly impossible due to time constraints and the ongoing COVID-19 pandemic would have no doubt affected the results since the

use of public transport heavily dropped [58, 71, 87, 39].

Collecting from other relevant locations. Collecting data for multiple relevant train stations increases the sample size. **However** data could not be used across stations, therefore an additional location feature would have to be introduced to create one overarching classifier. There are no doubt other stations situated next to a level crossing as there are approximately 7500 level crossings in Great Britain [79]. In this case, the manual measurements taken at the level crossing situated next to the station of Lincoln Central might not be entirely accurate for similar crossings.

Smaller class partitions. The classes for this problem are dependent on which interval the *length* feature falls in for the entry, but these are not placed at a uniform distance away as shown earlier. Therefore creating k classes, starting from $length = 0$ to $length = max$, each placed at an even distance i , might be a better model for predicting much better values. Even in this case, it is very likely that the data will be imbalanced, and thus some additional work may be needed to possibly drop some of the last classes and their observations completely.

Overcoming class imbalance. One issue which has been mentioned a lot is that classes are severely imbalanced, even in the data sets which are undersampled; there are Near-Miss algorithms that use a Nearest Neighbour classifier to select which samples to remove [66], amongst other intelligent undersampling methods [57]. Oversampling is similar, whereby random observations are duplicated in the minority group [97] or in the case of images, transformed; e.g. rotated, translated or otherwise transformed, or picking out specific features [76]. Oversampling has been shown to run the risk of overfitting [28] and so any form of oversampling must be carried out methodically [26, 47, 25].

Better manual readings. Another simple performance increase may occur by having more accurate data for the manual measurements. This could include either more randomly sampled readings throughout the day or readings taken at defined intervals every day. The latter might be better in the case that these values change throughout each day. Improved values will increase the efficiency of Algorithm 3 since with smaller t_w there will be more, shorter sessions; t_d could be modelled by a more accurate distribution and t_u would be a truer mean - all increasing the quality of the data. All of this could be improved if the data is collected directly on the crossing via template matching [24] or various tracking [31] methods.

Concept of freight trains. As described by Table 4.1, there are not many observations of cases where the level crossing was untraversable for longer than 300 seconds - in those cases, it was usually due to trains bunching up with late trains and causing a delay. An interesting aspect to include in the algorithm would have been freight trains, which passing by are typically slow and long. There is no publicly available information on freight trains but one way could have been to artificially "schedule" two freight trains [104] for a random time taken from an arbitrary time interval during a day.

Better feature engineering. Previously described points could increase the **quality** of the data, but other techniques remain available in terms of prepro-

cessing. Perhaps the most significant feature engineering technique that can be applied is the concept of larger shifts $\Downarrow_n$ - one could add additional $\Downarrow_n$ features for $\sin_a$ and $\cos_a$ as well as $|length|$ up until some arbitrary value which improves the model's memory. This was not investigated here in depth beyond a lag feature of 1, because each different model would have to be trained and cross validated individually, which takes on average 20-28 hours for each generation. It may be that there is correlation between an upcoming value $f(t)$ and an arbitrary κ number of previous values [85], but unfortunately this could not be tested.

Chapter 5

Conclusion

This paper produces a structured and systematic method for building a multi class time series predictor, containing approaches, models and the logical steps. There were two classifier models produced using each chosen supervised learning algorithm. Each learning algorithm was trained on the entire data set generated by the developed data simulating algorithm, and on a reduced, more balanced data set randomly sampled from the original data set to investigate the difference in performance. These models were ranked according to two different metrics, f1 macro average and f1 weighted average.

All models were considered according to each metric, but ultimately the models are ranked in accordance with their **macro average** scores, because the goal of this classifier is to recognise as many samples as possible across each class. The best performing classifier was Random Forest trained on the reduced data set, with 65%; followed by the other Random Forest model with 58%. Interestingly Nearest Neighbour trained on the full data set only did 1% worse, also being a close second with 79% weighted average. The reduced Nearest Neighbour model is followed by the ANN model at 44% macro average, failing to recognise any observations from c_4 . The second ANN trained on the full data set performed 4% worse, also failing to recognise any c_4 or c_3 observations. Whilst the f1 scores were relatively low in general, it was demonstrated that with more data it is possible to produce a reliable classifier which can predict level crossing closures in real-time with this method.

The performance was mostly lacking due to the unreliability and questionable integrity of the collected data; it was not possible to use sensors or a camera for data collection so the data quality could be improved, but good feature engineering is also vital. Other than increasing the data quality, some other techniques could have been used to exploit the full potential in the data, like multiple time shift features, more than four classes to try to balance data further or oversampling rare cases. These changes unfortunately turned out to be out of scope due to the training times of 20-28 hours per model.

In general, the project has been a success despite ways of improvements remaining uninvestigated due to time constraints; one major setback for the project was the inability of using computer vision techniques for data collection, which would have been the equivalent of a sensor and there would not be a reliance on the created algorithm for the data stream. The setback meant that some initial research and developed methods were no longer relevant to the project.

Generating a data stream required me capturing data on trains traveling to or from Lincoln Central, UK - the data collection was running from December, 2019 until March, 2020. There are 8640 records of the level crossing descending (and ascending) during this period, which was generated based on train data. There are 12172 captured trains traveling on this line during this period - 17.8% of the captured trains were **at least** one minute late, and 2.2% of all trains were more than 10 minutes late ¹; a 2.7% of all trains were cancelled, 28.4% of which were on sundays, whilst the remaining 71.6% of cancels were almost uniformly distributed between the remaining 6 days.

The overall conclusion is that implementing such a classifier model into popular navigation applications is possible; the method presented here is shown to be able to predict traversability based on timetabled data. The data stream simulating algorithm also supports additional time delays, for example extra time needed for a train to reach near a level crossing from a nearby station. I hope that readers are able to utilise some of the techniques outlined in this paper for tackling similar time series problems and that reading this paper has been enjoyable.

¹these are very similar findings to Shirvanyan's [91]

Chapter 6

Acknowledgements

I would like to express my greatest gratitude to Dr. Matthew Wattkins who has given me endless and invaluable guidance throughout the course of this project. I would also like to extend the thanks to each and every lecturer at the School of Mathematics & Physics who have not only taught but also inspired me during the past 3 years.

Last but not least, I want to thank my friends and family who listened to my endless rants and supported me every step of the way, not only during this project but during my time at university in general.

Chapter 7

Appendix

7.0.1 GitHub repository

All of my code and data can be found with additional information on this repository, should you choose to refer to some code examples: <https://github.com/peti123x/level-crossing-classifier>

7.0.2 Collected data

Table 7.1: Database structure for stored trains

Name	Data type
id	INT
origin	VARCHAR
destination	VARCHAR
due	TIME
delay	INT
est	TIME
cancelled	BOOLEAN
day	VARCHAR
date	DATE

Table 7.2: Time elapsed between barrier descending and train passing

Date	Time	Time elapsed (s)
28/12/19	16:41	112
30/12/19	12:53	62
11/12/19	13:28	23
13/12/19	21:49	34
14/12/19	14:51	74
15/12/19	13:23	33
16/12/19	17:47	18
18/12/19	11:03	17
19/12/19	19:33	82
22/12/19	12:17	47
27/12/19	11:19	81
03/01/20	16:54	19
04/01/20	20:42	33
06/01/20	21:35	111
07/01/20	14:28	35
11/01/20	17:42	66
14/01/20	11:54	42

Table 7.3: Seconds elapsed between trains without barrier changing state

Date	Time	Time elapsed (s)
13/12/19	21:47	56
13/12/19	21:50	105
13/12/19	21:54	84
15/12/19	13:24	32
17/12/19	16:54	25
18/12/19	12:53	230
19/12/19	19:35	72
27/12/19	11:21	95
06/01/20	21:38	63
06/01/20	11:47	49
09/01/20	14:04	54
11/01/20	15:56	89
11/01/20	17:44	95

Table 7.4: Time elapsed between train passing and barrier ascending

Date	Time	Time elapsed (s)
30/12/19	15:12	20
30/12/19	15:28	11
02/01/20	11:46	7
13/12/19	21:55	3
14/12/19	14:55	6
15/12/19	13:25	4
16/12/19	17:48	7
17/12/19	16:55	3
28/12/19	15:36	13
04/01/20	20:44	8
06/01/20	11:49	15
06/01/20	21:40	7
09/01/20	14:06	19
10/01/20	11:13	3
11/01/20	15:58	6
11/01/20	17:46	8

7.0.3 Research document

Investigating whether the use of Machine Learning can determine train arrivals and train-crossing barrier release

Research Plan

Description

Machine Learning provides systems the ability to learn and improve from data without being explicitly programmed. [1] Its application can help us identify everyday problems, like the average deviation of arrival each day of the week. This project will use sophisticated data analysis methods to determine when trains will arrive at the station to reroute pedestrians and drivers if they are likely to be late by waiting at the crossing barriers.

Sections in the report will mostly build on knowledge from previous sections. First the report introduces the sophisticated techniques used, image processing [2] and machine learning, and the mathematics behind them. Following this, a section focuses on explaining how these concepts are relevant to solving this problem. A method is then outlined to understand how data is collected for the Machine Learning algorithm(s), how the data is analysed to determine how late the next train is likely to be and how long the barrier is likely to be down for. The following section summarises the validity of achieved data and showcases the website implemented for telling when the next train is likely to arrive.

Reason

Late train arrivals can singlehandedly ruin someone's day especially when this train goes through a crossing unexpectedly, that is heavily used by both foot and driving traffic. Lincoln and its university are heavily affected by this issue, as there are dozens of trains passing through a crossing that is situated right at the bottom of its campus. While there was a bridge built recently to combat this issue, there are still a plethora of pedestrians who wait at the crossing with no information on when they will be able to use the crossing, which can be 5-10 minutes or more. This also causes lots of cars piling up on this street making traffic in the area worse.

Machine learning has always been an interest of mine and since its discovery I have been excited to utilise it for tackling a real-life problem. Seeing the local issues posed on foot traffic made me think whether machine learning can help identify a pattern in late train arrivals and provide a real-time estimation on how long the barriers will interrupt the flow of traffic.

Related Modules

Basic image processing techniques were taught in a C++ programming module, which provides a fundamental approach to tackling the data collection part of this project which will be done by using a microcomputer, a camera and some Python programming. Last year's Statistics module gives a great foundation to the data analysis aspect of this project, while the Machine Learning module I am currently enrolled in will teach me the next step up from the methods taught in Statistics. I have also

studied Artificial Intelligence in a range of modules which gives a bit more understanding to how machine learning operates and its algorithms and what data I should be aiming to process.

Survey of literature

First and foremost, I will need to read about setting up a Raspberry Pi. [3] The official documentation(s) are comprehensive which makes it easy and accessible to pick up.

For data collection it's important to assess available image processing Python libraries [4] and the technique of template matching [5], that allows me to search for a given object within an image. I also need to assess how rain drops, snow and lightning reflection may affect the matching algorithm and how to overcome these if needed. Before the collection begins, further reading is required into ML to have a vast understanding of how and what data needs storing to achieve my goal.

Equipment

The way data will be collected is that a camera will be pointed at the railway crossing from inside a building, which will take pictures every so often to monitor the barrier's movement. I plan to place the equipment in DCB3203 (Mrs Siobhan Wray's office), to which it would be useful to have access to so I am able to maintain should something go wrong.

To be able to do this, I will need the following:

- [Raspberry Pi 4 4GB \(£54\)](#); this is a powerful microcomputer with low energy consumption, which will allow continuous processing, logging and information mailing at the same time. It's important that it is energy efficient, as it is estimated that it will have to run 24/7 for about *2 months*.
- [15W USB-C power supply \(£8\)](#); to power the microcomputer. The only accessory that needs powering is the camera, so it is a good enough power supply.
- [microHDMI to HDMI cable \(£5\)](#); to connect to a HDMI display while development is ongoing.
- ['NOOBS' Preinstalled Micro SD card 32GB \(£9\)](#); an SD card that has the RaspberryPI Linux OS preinstalled. The newer version is now quite space heavy so the 32GB size makes sure I can take good quality pictures to allow for accurate image processing
- [Raspberry Pi Camera Module V2 \(£24\)](#); a camera native to the Raspberry PI API interface that allows for 1080p images.
- [Camera mount \(£3.50\)](#); so that camera can be placed securely against the window.

In total this will cost **£103.50**. I also need a USB driven **keyboard**, **mouse** and an HDMI display for development, but these probably do not need purchasing.

Risk Assessment

As this project has an experimental side to it, there were several risks I had to consider and a way to overcome them.

1. **The equipment may be stolen or tempered with.** I am making sure that it is placed in a restricted access office with CCTV.
2. **There may not be enough data collected.** The online timetable information may be scraped, and lateness is simulated by generating random numbers off a normal distribution curve. Adding a random number of arriving trains simulates freight trains passing through. This only will be done if there isn't enough collected data.

3. **Ethical issues regarding people on the images.** The computer will not process or log people, nor will it store or send the images taken.

Action Plan

W/c	Due	Action
07/10		Finish general research, get permission(s) for putting equipment in place
14/10	Research plan	Begin research on template matching
21/10	Logbook 1	
28/10		Setup equipment and develop Python code for data collection
04/11		
11/11		Begin data collection. Closely monitor the equipment and data for a week.
18/11		Specific research into Machine Learning
25/11	Logbook 2	
02/12		Write up image processing
09/12		
16/12		
23/12		Exam preparation. Potential time for more research or catch up.
30/12		
06/01		Exam week
13/01		Write up of Machine Learning
20/01	Logbook 3	
27/01		
03/02		Examine collected data and investigate its significance. Start training & data analysis
10/02		Write up method
17/02		Write up introduction and conclusion
24/02		
02/03		Brush up on sections ready for draft hand in.
09/03	Draft	Hand in draft
16/03		Create website.
23/03		Finish writing up sections and alter with information from feedback.
30/03		
06/04		
13/04		
20/04	Report hand-in	Create presentation and hand in as draft
27/04		Alter viva with information from draft
04/05	Viva	

Meetings

During our first meeting with my supervisor, we have agreed to meet every week on Mondays, at 1pm, unless otherwise agreed before. Any questions or work I need him to look at I will collect before the meeting takes place so that he has the chance to look at and consider these ahead of time.

References

- [1] Expert System. (2019). *What is Machine Learning? A definition - Expert System*. [online] Available at: <https://www.expertsystem.com/machine-learning-definition/> [Accessed 6 Oct. 2019].
- [2] Anbarjafari, G. (2019). *1. Introduction to image processing*. [online] Sisut.ut.ee. Available at: <https://sisu.ut.ee/imageprocessing/book/1> [Accessed 6 Oct. 2019].
- [3] Projects.raspberrypi.org. (2019). [online] Available at: <https://projects.raspberrypi.org/en/projects/raspberry-pi-getting-started> [Accessed 6 Oct. 2019].
- [4] Opensource.com. (2019). *10 Python image manipulation tools*. [online] Available at: <https://opensource.com/article/19/3/python-image-manipulation-tools> [Accessed 6 Oct. 2019].
- [5] Opencv-python-tutroals.readthedocs.io. (2019). *Template Matching — OpenCV-Python Tutorials 1 documentation*. [online] Available at: https://opencv-python-tutroals.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_template_matching/py_template_matching.html#template-matching [Accessed 6 Oct. 2019].

Bibliography

- [1] A. F. Agarap. Deep learning using rectified linear units (relu). *arXiv preprint arXiv:1803.08375*, 2018.
- [2] A. F. M. Agarap. A neural network architecture combining gated recurrent unit (gru) and support vector machine (svm) for intrusion detection in network traffic data. In *Proceedings of the 2018 10th International Conference on Machine Learning and Computing*, pages 26–30, 2018.
- [3] Y. Akbulut, A. Sengur, Y. Guo, and F. Smarandache. Ns-k-nn: Neutrosophic set-based k-nearest neighbors classifier. *Symmetry*, 9(9):179, 2017. doi: 10.3390/sym9090179.
- [4] A. Alalshekmubarak and L. S. Smith. A novel approach combining recurrent neural network and support vector machines for time series classification. In *2013 9th International Conference on Innovations in Information Technology (IIT)*, pages 42–47. IEEE, 2013.
- [5] B. Ambielli. Information entropy and information gain, 2017. URL <https://bambielli.com/til/2017-10-22-information-gain/>.
- [6] B. Ambielli. Gini impurity (with examples), 2017. URL <https://bambielli.com/til/2017-10-29-gini-impurity/>.
- [7] D. Anguita, A. Ghio, S. Ridella, and D. Sterpi. K-fold cross validation for error rate estimate in support vector machines. In *DMIN*, pages 291–297, 2009.
- [8] V. Asch. *Macro-and micro-averaged evaluation measures [[BASIC DRAFT]]*. 2013. URL <https://pdfs.semanticscholar.org/1d10/6a2730801b6210a67f7622e4d192bb309303.pdf>.
- [9] R. Bardenet, M. Brendel, B. Kégl, and M. Sebag. Collaborative hyperparameter tuning. In *International conference on machine learning*, pages 199–207, 2013.
- [10] R. Boostani, B. Graimann, M. Moradi, and G. Pfurtscheller. A comparison approach toward finding the best feature and classifier in cue-based bci. *Medical & biological engineering & computing*, 45(4):403, 2007.

- [11] M. Borowczak. Rpubs - benchmarking 20 machine learning models accuracy and speed, 2016. URL https://rpubs.com/m3cinc/Benchmarking_20_Machine_Learning_Models_Accuracy_and_Speed.
- [12] A. Bouchard-Côté. *Feature Engineering and Selection*. Berkeley University of California, 2009. URL <https://people.eecs.berkeley.edu/~jordan/courses/294-fall09/lectures/feature/slides.pdf>.
- [13] I. Bougoudis, K. Demertzis, and L. Iliadis. Hisycol a hybrid computational intelligence system for combined machine learning: the case of air pollution modeling in athens. *Neural Computing and Applications*, 27(5): 1191–1206, 2016.
- [14] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/a:1010933404324.
- [15] D. J. Briggs, C. de Hoogh, J. Gulliver, J. Wills, P. Elliott, S. Kingham, and K. Smallbone. A regression-based method for mapping traffic-related air pollution: application and testing in four contrasting urban environments. *Science of the Total Environment*, 253(1-3):151–167, 2000.
- [16] J. Brownlee. Discover feature engineering, how to engineer features and how to get good at it, 2014. URL <https://machinelearningmastery.com/discover-feature-engineering-how-to-engineer-features-and-how-to-get-good-at-it/>.
- [17] J. Brownlee. Supervised and unsupervised machine learning algorithms, 2016. URL <https://machinelearningmastery.com/supervised-and-unsupervised-machine-learning-algorithms/>.
- [18] J. Brownlee. Basic feature engineering with time series data in python, 2016. URL <https://machinelearningmastery.com/basic-feature-engineering-time-series-data-python/>.
- [19] J. Brownlee. How to one hot encode sequence data in python, 2017. URL <https://machinelearningmastery.com/how-to-one-hot-encode-sequence-data-in-python/>.
- [20] J. Brownlee. How much training data is required for machine learning?, 2017. URL <https://machinelearningmastery.com/much-training-data-required-machine-learning/>.
- [21] J. Brownlee. Bagging and random forest ensemble algorithms for machine learning, 2019. URL <https://machinelearningmastery.com/bagging-and-random-forest-ensemble-algorithms-for-machine-learning/>.
- [22] J. Brownlee. A gentle introduction to the rectified linear unit (relu) for deep learning neural networks, 2019. URL <https://machinelearningmastery.com/rectified-linear-activation-function-for-deep-learning-neural-networks/>.

- [23] J. Brownlee. Why one-hot encode data in machine learning?, 2019. URL <https://machinelearningmastery.com/why-one-hot-encode-data-in-machine-learning/>.
- [24] R. Brunelli. *Template matching techniques in computer vision: theory and practice*. John Wiley & Sons, 2009.
- [25] C. Bunkhumpornpat, K. Sinapiromsaran, and C. Lursinsap. Safe-level-smote: Safe-level-synthetic minority over-sampling technique for handling the class imbalanced problem. In *Pacific-Asia conference on knowledge discovery and data mining*, pages 475–482. Springer, 2009.
- [26] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer. Smote: synthetic minority over-sampling technique. *Journal of artificial intelligence research*, 16:321–357, 2002.
- [27] N. V. Chawla, N. Japkowicz, and A. Kotcz. Editorial. *ACM SIGKDD Explorations Newsletter*, 6(1):1, 2004. doi: 10.1145/1007730.1007733.
- [28] N. V. Chawla, N. Japkowicz, and A. Kotcz. Special issue on learning from imbalanced data sets. *ACM SIGKDD explorations newsletter*, 6(1):1–6, 2004.
- [29] F. Chollet et al. Keras. <https://keras.io>, 2015.
- [30] R. B. Cleveland. Stl: A seasonal-trend decomposition procedure based on loess, 1990. URL <https://www.scb.se/contentassets/ca21efb41fee47d293bbee5bf7be7fb3/stl-a-seasonal-trend-decomposition-procedure-based-on-loess.pdf>.
- [31] B. Coifman, D. Beymer, P. McLauchlan, and J. Malik. A real-time computer vision system for vehicle tracking and traffic surveillance. *Transportation Research Part C: Emerging Technologies*, 6(4):271–288, 1998.
- [32] C. Cortes, M. Mohri, and A. Rostamizadeh. L2 regularization for learning kernels. *arXiv preprint arXiv:1205.2653*, 2012.
- [33] S. Cramer, M. Kampouridis, A. A. Freitas, and A. K. Alexandridis. An extensive evaluation of seven machine learning methods for rainfall prediction in weather derivatives. *Expert Systems with Applications*, 85:169–181, 2017.
- [34] P. Cseik. level-crossing-classifier, 2020. URL <https://github.com/peti123x/level-crossing-classifier>.
- [35] P. Cunningham and S. J. Delany. k-nearest neighbour classifiers-. *arXiv preprint arXiv:2004.04523*, 2020.
- [36] T. Dietterich. Overfitting and undercomputing in machine learning. *ACM computing surveys (CSUR)*, 27(3):326–327, 1995.

- [37] P. Domingos. A few useful things to know about machine learning. *Communications of the ACM*, 55(10):78, 2012. doi: 10.1145/2347736.2347755.
- [38] S. S. Du, X. Zhai, B. Póczos, and A. Singh. Gradient descent provably optimizes over-parameterized neural networks. *arXiv preprint arXiv:1810.02054*, 2018.
- [39] D. C. E. editor. Uk road travel falls to 1955 levels as covid-19 lockdown takes hold. *The Guardian*, 2020. URL <https://www.theguardian.com/uk-news/2020/apr/03/uk-road-travel-falls-to-1955-levels-as-covid-19-lockdown-takes-hold-coronavirus-traffic>.
- [40] D. Ellerman. Logical entropy: Introduction to classical and quantum logical information theory. *Entropy*, 20(9):679, 2018.
- [41] M. H. Esfe, M. Afrand, S. Wongwises, A. Naderi, A. Asadi, S. Rostami, and M. Akbari. Applications of feedforward multilayer perceptron artificial neural networks and empirical correlation for prediction of thermal conductivity of mg (oh) 2-eg using experimental data. *International Communications in Heat and Mass Transfer*, 67:46–50, 2015.
- [42] P. A. Flach. The geometry of roc space: understanding machine learning metrics through roc isometrics. In *Proceedings of the 20th international conference on machine learning (ICML-03)*, pages 194–201, 2003.
- [43] M. B. Fraj. In depth: Parameter tuning for random forest, 2017. URL <https://medium.com/all-things-ai/in-depth-parameter-tuning-for-random-forest-d67bb7e920d>.
- [44] gung Reinstate Monica (<https://stats.stackexchange.com/users/7290/gung-reinstate-monica>). Why are bias nodes used in neural networks? Cross Validated, 2017. URL <https://stats.stackexchange.com/q/186284>. URL:<https://stats.stackexchange.com/q/186284> (version: 2017-06-29).
- [45] D. Gupta. Activation functions — fundamentals of deep learning, 2020. URL <https://www.analyticsvidhya.com/blog/2020/01/fundamentals-deep-learning-activation-functions-when-to-use-them/>.
- [46] P. Gupta. Cross-validation in machine learning, 2017. URL <https://towardsdatascience.com/cross-validation-in-machine-learning-72924a69872f>.
- [47] H. Han, W.-Y. Wang, and B.-H. Mao. Borderline-smote: a new over-sampling method in imbalanced data sets learning. In *International conference on intelligent computing*, pages 878–887. Springer, 2005.
- [48] H. He and E. A. Garcia. Learning from imbalanced data. *IEEE Transactions on knowledge and data engineering*, 21(9):1263–1284, 2009.

- [49] J. Heaton. The number of hidden layers — heaton research, 2008. URL <https://web.archive.org/web/20140721050413/http://www.heatonresearch.com/node/707>.
- [50] J. Heaton. An empirical analysis of feature engineering for predictive modeling. In *SoutheastCon 2016*, pages 1–6. IEEE, 2016.
- [51] hobs (<https://stats.stackexchange.com/users/15974/hobs>). How to choose the number of hidden layers and nodes in a feedforward neural network? Cross Validated, 2019. URL <https://stats.stackexchange.com/q/136542>. URL:<https://stats.stackexchange.com/q/136542> (version: 2019-05-27).
- [52] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [53] M. Hossin and M. Sulaiman. A review on evaluation metrics for data classification evaluations. *International Journal of Data Mining & Knowledge Management Process*, 5(2):1, 2015.
- [54] S. K. (<https://stats.stackexchange.com/users/1352/stephan-ko-lassa>). scale a number between a range. Cross Validated, 2018. URL <https://stats.stackexchange.com/q/281164>. URL:<https://stats.stackexchange.com/q/281164> (version: 2018-04-19).
- [55] Z. (<https://stats.stackexchange.com/users/2817/zach>). Optimal construction of day feature in neural networks. Cross Validated, 2014. URL <https://stats.stackexchange.com/q/126234>. URL:<https://stats.stackexchange.com/q/126234> (version: 2014-12-02).
- [56] E. Jang, S. Gu, and B. Poole. Categorical reparameterization with gumbel-softmax. *arXiv preprint arXiv:1611.01144*, 2016.
- [57] J. M. Johnson and T. M. Khoshgoftaar. Survey on deep learning with class imbalance. *Journal of Big Data*, 6(1), 2019. doi: 10.1186/s40537-019-0192-5.
- [58] J. Jolly. Bailout expected for uk city transport networks crippled by covid-19, 2020. URL <https://www.theguardian.com/uk-news/2020/apr/22/bailout-expected-for-uk-city-transport-networks-crippled-by-covid-19>.
- [59] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization, 2014. URL <https://arxiv.org/abs/1412.6980>.
- [60] W. Koehrsen. Hyperparameter tuning the random forest in python, 2018. URL <https://towardsdatascience.com/hyperparameter-tuning-the-random-forest-in-python-using-scikit-learn-28d2aa77dd74>.
- [61] A. Krizhevsky, I. Sutskever, and G. E. Hinton. *ImageNet Classification with Deep Convolutional Neural Networks*. University of Toronto, 2012. URL <http://www.cs.toronto.edu/~fritz/absps/imagenet.pdf>.

- [62] M. T. Leung, H. Daouk, and A.-S. Chen. Forecasting stock indices: a comparison of classification and level estimation models. *International Journal of Forecasting*, 16(2):173–190, 2000.
- [63] A. Liaw, M. Wiener, et al. Classification and regression by randomforest. *R news*, 2(3):18–22, 2002.
- [64] I. London. Encoding cyclical continuous features - 24-hour time, 2016. URL <https://ianlondon.github.io/blog/encoding-cyclical-features-24hour-time/>.
- [65] Y. Low, J. Gonzalez, A. Kyrola, D. Bickson, C. Guestrin, and J. M. Hellerstein. Distributed graphlab: A framework for machine learning in the cloud. *arXiv preprint arXiv:1204.6078*, 2012.
- [66] I. Mani and I. Zhang. knn approach to unbalanced data distributions: a case study involving information extraction. In *Proceedings of workshop on learning from imbalanced datasets*, volume 126, 2003.
- [67] R. Mitchell. *Web scraping with Python: Collecting more data from the modern web.* ” O’Reilly Media, Inc.”, 2018.
- [68] Moefasamoefasa. Why don’t tree ensembles require one-hot-encoding?, 2017. URL <https://datascience.stackexchange.com/a/18058>.
- [69] M. N. Murty and V. S. Devi. *Pattern Recognition: an Algorithmic Approach*. Springer, 2011.
- [70] NIST/SEMATECH, 2012. URL <https://www.itl.nist.gov/div898/handbook/eda/section3/eda366h.htm>.
- [71] J. O’Rourke. Passenger data indicates impact of coronavirus on uk rail in, 2020. URL <https://www.transportextra.com/publications/local-transport-today/news/64887/passenger-data-indicates-impact-of-coronavirus-on-uk-rail-industry/>.
- [72] S. Panchapagesan, M. Sun, A. Khare, S. Matsoukas, A. Mandal, B. Hoffmeister, and S. Vitaladevuni. Multi-task learning and weighted cross-entropy for dnn-based keyword spotting. In *Interspeech*, volume 9, pages 760–764, 2016.
- [73] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: K nearest neighbours classifier, 2019. URL <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>.

- [74] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Neural network models (supervised), 2019. URL https://scikit-learn.org/stable/modules/neural_networks_supervised.html.
- [75] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Random forest classifier, 2019. URL <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>.
- [76] F. Pernici and A. Del Bimbo. Object tracking by oversampling local features. *IEEE transactions on pattern analysis and machine intelligence*, 36(12):2538–2551, 2013.
- [77] D. M. Powers. Evaluation: from precision, recall and f-measure to roc, informedness, markedness and correlation. 2011.
- [78] P. Probst, M. N. Wright, and A.-L. Boulesteix. Hyperparameters and tuning strategies for random forest. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 9(3):e1301, 2019.
- [79] I. Prosser. Level crossings — office of rail and road, 2017. URL <https://orr.gov.uk/rail/health-and-safety/infrastructure-safety/level-crossings>.
- [80] pythiest (<https://datascience.stackexchange.com/users/40949/pythiest>). Micro average vs macro average performance in a multiclass classification setting. Data Science, 2017. URL <https://datascience.stackexchange.com/a/24051/87912>. URL:<https://datascience.stackexchange.com/a/24051/87912> (version: 2017-10-25).
- [81] N. Rail, 2015. URL https://www.nationalrail.co.uk/times_fares/ldb.aspx.
- [82] L. E. Raileanu and K. Stoffel. Theoretical comparison between the gini index and information gain criteria, 2004. URL https://www.unine.ch/files/live/sites/imi/files/shared/documents/papers/Gini_index_fulltext.pdf.
- [83] B. Ricaud. A simple explanation of entropy in decision trees, 2017. URL <https://bricaud.github.io/personal-blog/entropy-in-decision-trees/>.

- [84] S. D. Sawaitul, K. Wagh, and P. Chatur. Classification and prediction of future weather by using back propagation algorithm-an approach. *International Journal of Emerging Technology and Advanced Engineering*, 2 (1):110–113, 2012.
- [85] S. says. Feature engineering techniques for time series data, 2019. URL <https://www.analyticsvidhya.com/blog/2019/12/6-powerful-feature-engineering-techniques-time-series/>.
- [86] C. Schaffer. Selecting a classification method by cross-validation. *Machine Learning*, 13(1):135–143, 1993.
- [87] R. Schraer. Coronavirus: What’s the risk of flying or taking the train? *BBC*, 2020. URL <https://www.bbc.co.uk/news/health-51736185>.
- [88] G. Seif. Why and how to do cross validation for machine learning, 2019. URL <https://towardsdatascience.com/why-and-how-to-do-cross-validation-for-machine-learning-d5bd7e60c189>.
- [89] J. Sen. *Chapter 5: A Time Series Analysis-Based Forecasting Framework for the Indian Healthcare Sector*, pages 134 – 161. 2017. ISBN 978-3-330-65386-3.
- [90] Y. Seth. Which activation function to use in neural networks?, 2018. URL <https://yashuseth.blog/2018/02/11/which-activation-function-to-use-in-neural-networks/>.
- [91] L. Shirvanyan. *M1 Project: Railway Delay Statistics and Modelling*. University of Warwick, 2015. URL https://warwick.ac.uk/fac/cross_fac/complexity/study/emmc/outcomes/studentprojects/shirvanyan.pdf.
- [92] G. Singh and R. K. Panda. Daily sediment yield modeling with artificial neural network using 10-fold cross validation method: a small agricultural watershed, kapgari, india. *International Journal of Earth Sciences and Engineering*, 4(6):443–450, 2011.
- [93] M. Sokolova and G. Lapalme. A systematic analysis of performance measures for classification tasks. *Information processing & management*, 45 (4):427–437, 2009.
- [94] M. Sundermeyer, H. Ney, and R. Schluter. From feedforward to recurrent lstm neural networks for language modeling. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 23(3):517–529, 2015. doi: 10.1109/taslp.2015.2400218. URL <https://pdfs.semanticscholar.org/f9a1/b3850dfd837793743565a8af95973d395a4e.pdf>.
- [95] K. Teknomo. Knn numerical example (hand computation), 2017. URL https://people.revoledu.com/kardi/tutorial/KNN/KNN_Numerical-example.html.

- [96] P. Thanh Noi and M. Kappas. Comparison of random forest, k-nearest neighbor, and support vector machine classifiers for land cover classification using sentinel-2 imagery. *Sensors (Basel, Switzerland)*, 18(1):18, 2017. doi: 10.3390/s18010018. URL <https://www.ncbi.nlm.nih.gov/pubmed/29271909>.
- [97] J. Van Hulse, T. M. Khoshgoftaar, and A. Napolitano. Experimental perspectives on learning from imbalanced data. In *Proceedings of the 24th international conference on Machine learning*, pages 935–942, 2007.
- [98] A. Vehtari, A. Gelman, and J. Gabry. Practical bayesian model evaluation using leave-one-out cross-validation and waic. *Statistics and computing*, 27(5):1413–1432, 2017.
- [99] A. Vidhya. Improve your model performance using cross validation (in python / r), 2018. URL <https://www.analyticsvidhya.com/blog/2018/05/improve-model-performance-cross-validation-in-python-r/>.
- [100] A. Vidhya. Tune a random forest model’s parameters for machine learning, 2019. URL <https://www.analyticsvidhya.com/blog/2015/06/tuning-random-forest-model/>.
- [101] A. S. Walia. Activation functions and it’s types-which is better?, 2017. URL <https://web.archive.org/web/20190726154104/https://towardsdatascience.com/activation-functions-and-its-types-which-is-better-a9a5310cc8f?gi=7baaa7574716>.
- [102] D. Wang. Stuck in traffic?, 2007. URL <https://googleblog.blogspot.com/2007/02/stuck-in-traffic.html>.
- [103] F. Wang, J. Cheng, W. Liu, and H. Liu. Additive margin softmax for face verification. *IEEE Signal Processing Letters*, 25(7):926–930, 2018.
- [104] W. S. Webb. *Supporting Lincolnshire Railways*. 2010. URL <https://lincolnshire.moderngov.co.uk/Data/Executive%20Councillor%20-%20Highways%20and%20Transport/20100504/Agenda/01740%20-%20County%20Rail%20Strategy%20-%20REPORT%20APPENDIX%20A.pdf>.
- [105] E. W. Weisstein. Beta distribution. URL <https://mathworld.wolfram.com/BetaDistribution.html>.
- [106] Y. Wu, J. Li, Y. Kong, and Y. Fu. Deep convolutional neural network with independent softmax for large scale face recognition. In *Proceedings of the 24th ACM international conference on Multimedia*, pages 1063–1067, 2016.

- [107] D.-F. Xia, S.-L. Xu, and F. Qi. A proof of the arithmetic mean-geometric mean-harmonic mean inequalities. *RGMIA research report collection*, 2 (1), 1999.
- [108] W. Xiang, H.-D. Tran, and T. T. Johnson. Reachable set computation and safety verification for neural networks with relu activations. *arXiv preprint arXiv:1712.08163*, 2017.
- [109] Z. Zabokrtsky. Wayback machine, 2016. URL https://web.archive.org/web/20160304112056/https://ufal.mff.cuni.cz/~zabokrtsky/courses/npfl1104/html/feature_engineering.pdf.
- [110] C. Zhou, C. Sun, Z. Liu, and F. Lau. *A C-LSTM Neural Network for Text Classification*. 2015. URL <https://arxiv.org/pdf/1511.08630.pdf>.